

HOCHSCHULE FÜR ANGEWANDTE WISSENSCHAFTEN
MÜNCHEN

FAKULTÄT FÜR INFORMATIK UND MATHEMATIK



Masterthesis
for obtaining the academic degree of
Master of Science
in course of studies Computer Science

Analysis of the PKI Ecosystem

Analyse des PKI Ökosystems

Author:	Erik Sirch
Matriculation number:	42971920
Submission Date:	08. February 2026
Supervisor:	Prof. Michael Heintl
Advisor:	Prof. Michael Heintl

Erklärung

Hiermit erkläre ich, dass ich die Masterarbeit selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle übernommenen Inhalte sowie mit Unterstützung von KI generierten Inhalte wurden entsprechend den anerkannten wissenschaftlichen Grundsätzen oder entsprechend der Regelungen zur Kennzeichnung von KI-Inhalten kenntlich gemacht. Ausgenommen von der Kenntlichmachung sind orthografische oder grammatikalische Korrekturen, Übersetzungen sowie nicht-sinnverändernde Verbesserungen von Formulierungen. Ich bin mir bewusst, dass mit KI generierte Texte keine Garantie für die Qualität von Inhalten und Text bieten. Daher erkläre ich, dass ich KI-Werkzeuge lediglich als Hilfsmittel genutzt habe, die von KI generierten Inhalte kritisch überprüft habe und mein eigenständiger kognitiver sowie kreativer Einfluss in dieser Arbeit überwiegt. Ich versichere, dass ich die Inhalte meiner Arbeit vollständig verstanden habe und selbstständig vertreten kann. Ich versichere, dass ich ausschließlich KI-Werkzeuge verwendet habe, deren Nutzung vom Prüfer oder der Prüferin als Hilfsmittel zugelassen wurden.

München, 07. Februar 2026

Erik Sirch

Acknowledgments

I sincerely thank Professor Michael Heint for his support and guidance throughout this thesis.

My sincere thanks also go to Maksym Vasylenko for providing me with computing resources beyond the default use cases. I also thank Professor Johannes Ebke for offering the HM-Kube platform and technical support, even on short notice.

I also express my gratitude to the people responsible for the researcher access program at *Censys*. By offering access to their extensive data source free of charge, they enabled me to perform an empirical analysis over billions of entries.

Abstract

Public Key Infrastructures (PKIs) underpin most of today’s secure communication, but the actual deployment characteristics of the ecosystem and its readiness for Post-Quantum Cryptography (PQC) remain poorly analyzed. This thesis presents *PKI Explorer*, a reusable measurement platform for large-scale empirical analysis of the Public-Key Infrastructure using X.509 (PKIX). *PKI Explorer* processes raw certificate data from Censys as of June 1, 2025, storing approximately 15 billion X.509v3 certificates in a column-oriented database and evaluating them along key usage, extended key usage, public key and signature algorithms (including PQC and hybrid extensions), and metadata such as trust status, origin, and locality.

The analysis addresses three research questions on (1) dominant certificate use cases, (2) current algorithm trends, and (3) the adoption of PQC algorithms in PKIX. For RQ1, the results show that leaf certificates are overwhelmingly dominated by the Web PKI profile: `digitalSignature` and `keyEncipherment` key usages combined with `serverAuth` extended key usage, indicating classic TLS client–server deployments. For RQ2, RSA with SHA-256 remains the dominant cryptographic system across all certificate types, with ECC as the distant second. For RQ3, no PQC algorithms were found in the currently trusted certificates observed by *Censys*. All detected certificates using PQC algorithms appear exclusively in certificates marked as not trustworthy, likely from test environments. Within the limitations of the scope observed by *Censys* and a single time snapshot, these findings indicate that large-scale adoption of PQC in productive PKIX ecosystems had not begun until June 01, 2025.

Contents

1	Introduction	1
2	Background	3
2.1	Public Key Infrastructure	3
2.1.1	Cryptography	3
2.1.2	Public Key Certificate	6
2.1.3	PKI Hierarchy	6
2.1.4	Public Trust vs. Private Trust	7
2.1.5	Certificate Revocation List and OCSP	7
2.2	Public Key Cryptography Standard	8
2.3	Certificate Transparency	8
2.4	Post-Quantum Cryptography	10
2.4.1	PQC Algorithms	10
2.4.2	PQC Integration Types	11
2.5	Column-oriented Databases	13
3	Related Work	15
3.1	Measurement of PKIX	15
3.2	Incident Studies on PKIX	15
3.3	Certificate Transparency Honeypot Analysis	16
3.4	IoT and Constrained Environments	16
3.5	Alternatives to PKIX	16
3.6	Analysis of PKIX	17
3.7	Post-Quantum Cryptography	17
3.8	Summary	18
4	Concept	19
4.1	Empirical Analysis	19
4.1.1	Certificate Usage Analysis	19
4.1.2	Algorithm Analysis	19
4.1.3	Metadata Filter	20
4.2	Data Source	21
4.2.1	Indexing Services	21
4.2.2	Firmware Images	23
4.2.3	Data Definition	23
5	Implementation	24
5.1	Architecture	24
5.2	Data Storing	24
5.2.1	Installation	25
5.2.2	Data Schema	26

5.2.3	Tools and Integration	27
5.3	Data Processing	27
5.3.1	Worker Implementation	28
5.3.2	Monitoring	29
5.4	Data Evaluation	29
5.5	Visualization	31
5.5.1	Backend-Proxy	31
5.5.2	Frontend Implementation	32
5.6	Trouble Shooting and Lessons Learned	32
5.6.1	Hardware Issues	34
5.6.2	Software Issues	34
6	Analysis	36
6.1	General Overview	36
6.1.1	Root Certificates	36
6.1.2	Intermediate Certificates	37
6.1.3	Leaf Certificates	38
6.1.4	Unknown Certificates	39
6.1.5	Summary	40
6.2	Key Usage Distribution	40
6.2.1	Root and Intermediate CAs	41
6.2.2	Leaf Certificates	42
6.2.3	Unknown Certificates	42
6.2.4	Summary	43
6.3	Extended Key Usage Distribution	43
6.3.1	Root and Intermediate CAs	43
6.3.2	Leaf Certificates	43
6.3.3	Unknown Certificates	44
6.3.4	Summary	44
6.4	Public Key Algorithm Distribution	45
6.4.1	General Algorithm Distribution	45
6.4.2	Distribution of PQC Algorithms	46
6.4.3	Summary	46
6.5	Signature Algorithm Distribution	47
6.5.1	General Algorithm Distribution	47
6.5.2	Distribution of PQC Algorithms	47
6.5.3	Summary	47
6.6	Alternative Signature Algorithm Distribution	48
6.7	Alternative Public Key Algorithm Distribution	48
6.8	Corrupted Certificates	49
7	Discussion	52
7.1	Data Quality	52
7.2	Use Case Trends	52

7.3	Algorithm Trends	53
7.3.1	PQC Adoption	53
8	Conclusion & Future Work	54
	List of Figures	56
	List of Listings	58
	List of Tables	59
	Bibliography	60
	Glossary	66
A	RFC 5280 Definitions	68
B	X.509v3 Definitions	70
C	OID Mapping	71
D	Code Excerpts	75

1 Introduction

PKI is a foundational building block of today's digital economy, enabling authentication, confidentiality, and integrity for the vast majority of the digital world. At the same time, advances in quantum computing threaten the hardness assumptions of currently deployed implementations. Studies predict that Cryptographically Relevant Quantum Computers (CRQCs) will be capable of breaking widely deployed cryptographic schemes and systems such as Rivest-Shamir-Adleman (RSA) and Elliptic Curve Cryptography (ECC). This directly affects the secure distribution of cryptographic keys for common security protocols such as TLS or S/MIME. Because digital certificates and identity documents often need to remain secure for many years, adversaries can already record encrypted traffic today and decrypt it once quantum capabilities become available, also known as the harvest now, decrypt later (HNDL) problem. In combination with the rapidly growing Internet, there is an urgent need to understand how Internet-facing PKI ecosystems, such as PKIX, are evolving and how quickly they are transitioning to PQC.

In response to this threat, the National Institute of Standards and Technology (NIST) standardized three sets of PQC algorithms in 2024. Those are cleared for immediate usage, initiating the transition towards quantum secure PKI ecosystems. Besides providers of PKI implementations, such as Let's Encrypt or Keyfactor, other companies and even nations have started the transition. Germany, for example, announced that its next generation of identification documents with online functionality will be secured using PQC algorithms¹.

Despite these developments, there are still limited empirical data on how PKIX is actually implemented at scale in different ecosystems. Especially to what extent PQC algorithms or hybrid constructions are already being adopted. Therefore, a systematic empirical analysis of PKIX implementations can provide essential guidance to security experts, vendors, and policymakers who seek to improve the security of future PKIX ecosystems.

To address this gap, this thesis introduces an analysis tool named *PKI Explorer*. *PKI Explorer* is designed to collect and analyze PKIX implementations on a large scale across multiple ecosystems. In doing so, this thesis has a particular focus on cryptographic algorithms, key usages, and transition mechanisms relevant to PQC. The empirical results generated with *PKI Explorer* will be used to answer the following research questions:

- RQ1: Which certificate use cases are the most represented in PKIX implementations?
- RQ2: What are the current trends for cryptographic algorithms in PKIX implementations?
- RQ3: What is the current adoption rate of PQC algorithms in PKIX implementations?

PKI Explorer will provide structured data on various certificate characteristics, allowing a detailed analysis of PKIX usage patterns and emerging PQC transition trends.

¹Press Release by BSI on quantum-safe identification documents: <https://www.bundesdruckerei.de/en/newsroom/press-releases/deutschland-setzt-massstabe-fur-sichere-ausweisdokumente-im-zeitalter-der-quantencomputer>

The remainder of this thesis is structured as follows. Chapter 2 presents the necessary background on PKIX, classical public-key cryptography, and PQC standards. Chapter 3 surveys related work on PKIX measurement and analysis, and PQC transition strategies. Chapter 4 details the concept of this thesis, laying the foundation for data sources and evaluation strategies. Chapter 5 deals with the technology behind *PKI Explorer*. In addition to the technical description, the development including the occurred problems will be described. Chapter 6 presents the empirical results generated by *PKI Explorer*. Therefore, providing the necessary data to answer the research questions. Afterwards, chapter 7 will compare the findings with results from similar works to reinforce or criticize them. Lastly, Chapter 8 summarizes the findings, describes limitations, and identifies directions for future research on quantum-resilient PKIX ecosystems.

2 Background

This chapter establishes the technical foundation needed during this thesis. First, the structural and operational aspects of PKI and X.509 certificates are explained. The following sections describe two evolutionary steps of PKIX: Certificate Transparency (CT) and PQC. The last section is an introduction to column-oriented databases, which are crucial technology when it comes to performing large-scale empirical analyses over tens of billions of certificates.

2.1 Public Key Infrastructure

PKI is:

The infrastructure able to support the management of public keys able to support authentication, encryption, integrity or non-repudiation services [27].

This vague and highly abstract definition has led to a wide range of different implementations. One of the most prominent variants of PKIs is PKIX. It is a standard that focuses on the needs of PKI in the Internet. This implementation is based on the recommendations published by the ITU X.509 standard [27], especially the concepts of X.509v3 certificates[46]. The general standard is maintained by the PKIX working group and publishes various standards such as RFC 5280 [11] or RFC 6960 [52]. Alternative concepts like OpenPGP [63] or SPKI [19] are different approaches to fulfill the needs of a PKI and are implemented in certain use-cases and environments. However, this thesis focuses on the concepts defined by PKIX and ITU X.509.

As ITU X.509 mentions, a set of concepts is needed to fully support the requirements of a PKI, which will be described in the following sections.

2.1.1 Cryptography

To protect data transmitted via public media (e.g., cable or satellite) from threats with bad intentions, it needs to be encrypted. Conventional public-key cryptosystems are based on mathematical problems that are believed to be hard on average, which means that typical instances cannot be solved in polynomial time with known algorithms. This makes such problems ideal for widespread secure usage. However, such mathematical problems need a second characteristic. An average problem shall only be hard to solve if certain information is unknown. This information is mostly referred to as a “secret key”. [29]

Symmetric Encryption

The first and simpler set of cryptographic algorithms is symmetric encryption. Figure 2.1 illustrates how a shared secret key is used to transform a text into an encrypted message that can only be decrypted to the original text using the same secret key. Examples of

symmetric encryption and decryption algorithms include Advanced Encryption Standard (AES), Data Encryption Standard (DES), and Escrowed Encryption Standard (EES). The simplicity of this approach comes with the catch that the shared secret key has to be distributed to all participating parties without being compromised, which is known as the key exchange problem. [3]

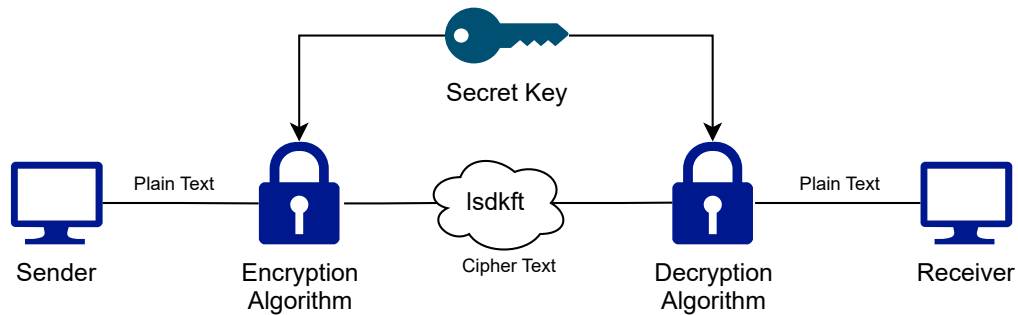


Figure 2.1: *Process of symmetric encryption. Adapted from Mohammed Alenezi et al. [3]*

Public Key Cryptography / Asymmetric Encryption

Asymmetric Encryption uses a different concept, generating a key pair. The first key is also called “Secret Key” and therefore should be kept private. The second key is called a “Public Key” and can be shared with anyone. Depending on how the key pair is generated, different use cases can be derived. First, Figure 2.2 shows how such a key pair can be used for encryption. The public key is used to encrypt the data. The encrypted data are sent to the owner of the key pair, who can decrypt the data using the private key. However, the performance of symmetric encryption is better than that of asymmetric encryption. Therefore, asymmetric encryption is often used to establish session keys, during the so called key establishment. After the exchange, both parties switch to symmetric encryption. SP 800-56A [7] and SP 800-56B [8] standardized which cryptographic systems and schemes can be used for key establishment and currently contain: RSA, Diffie-Hellman (DH), and Menezes-Qu-Vanstone (MQV).

However, this approach alone is vulnerable to man-in-the-middle (MITM) attacks [15]. In doing so, a malicious actor intercepts the communication between two parties blocking the public key exchange and distributes its own public key. Both parties establish a seemingly secure connection with the attacker and start the intended information exchange, which is decrypted by the attacker and re-encrypted before sending it to the other party.

Therefore, another concept is needed to authenticate whether a message has been altered. Public key cryptography enables this concept, because the secret key is an ideal basis for an authentication mechanism. Figure 2.3 shows how to create a signature for some data using the secret key. After that, the public key can be used to verify that the clear text and signature match. Thus, ensuring that the information has not been altered and originates from the party whose public key was used. This concept is called digital signatures and is standardized in FIPS 186-5 [38], which currently approves RSA, Elliptic Curve Digital

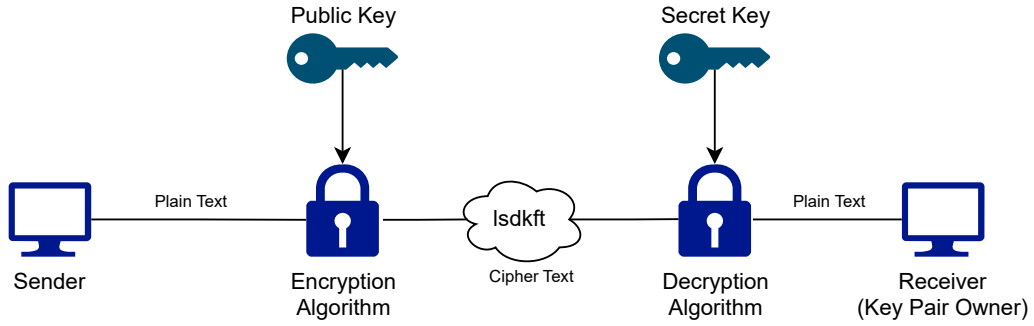


Figure 2.2: Process of asymmetric encryption. Adapted from Mohammed Alenezi et al. [3]

Signature Algorithm (ECDSA), and Edwards-Curve Digital Signature Algorithm (EdDSA) for the creation of new digital signatures.

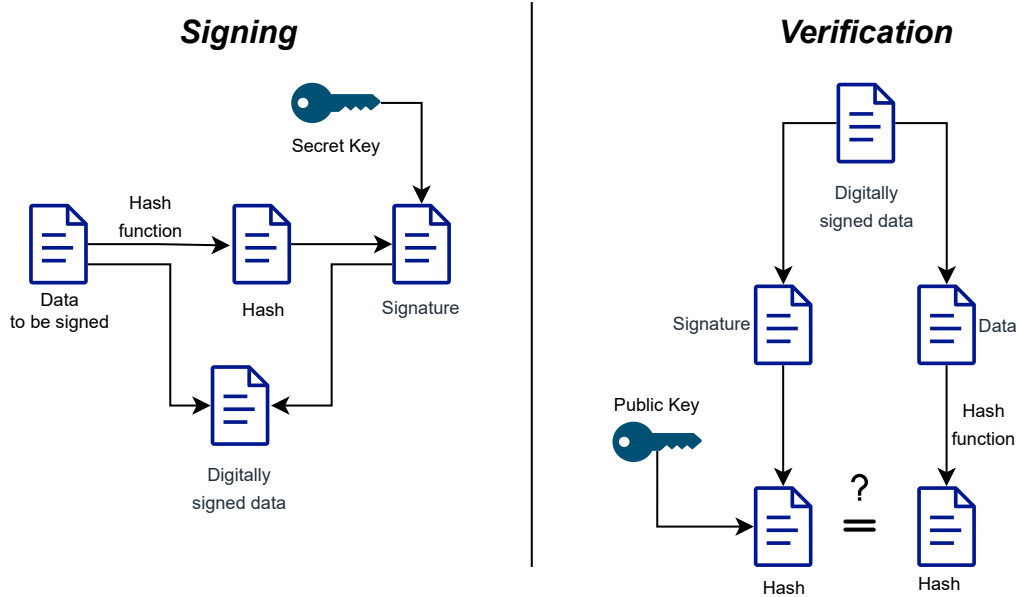


Figure 2.3: Process of digital signing and verification

Key Size and Algorithm Security

The SP 800-57 [5] defines the actual security strength of a specific algorithm in bits, with the possible values [80, 112, 128, 192, 256]. The values are derived from symmetric algorithms, since to break such an algorithm an exhaustion attack has to be executed. This means that every possible key has to be tried. Therefore, a symmetric algorithm that uses a key with a size of 128 bits can have 2^{128} possible keys, resulting in a strength of 128 bits. Therefore, when defining which algorithm was used for a key pair, it is accompanied

Attribute	Description
Version	Distinction between the three versions of X.509 V1, V2, and V3
Serial Number	Unique identifier assigned by the issuer to ensure uniqueness between all their signed certificates
Signature	OID of the algorithm used to sign this certificate
Issuer	DN of the issuing entity (cf. Table A.1)
Subject	DN of the certificate entity (cf. Table A.1)
Validity	Period of time, when the certificate is valid
Subject Public Key	The actual value of the public key and OID of the used algorithm
Extensions	A list of extensions enhancing a certificate with additional information (see Table A.2)

Table 2.1: *Key Attributes of a X.509v3 certificate and purpose description, derived from ITU X509 [27]*

by the key size used. So, if a certain key size for an algorithm turns out to be equally secure as AES-128, both receive the same level of security. Thus, enabling comparability between different types of algorithms. For example, the strength of AES using a 128-bit key is equal to ECDSA using a 256-bit key and RSA using a 3072-bit key. Since AES is the reference point, all three combinations have a security strength of 128 bits. Furthermore, SP 800-131A [9] defines a minimal security strength to be used for federal applications of currently 112 bits. However, a current draft of SP 800-131A Revision 3 [6] aims to increase this minimum to 128 bits until 2031.

2.1.2 Public Key Certificate

A public key certificate, specifically a X.509 certificate, is a data structure that contains information about an entity and the corresponding public key. The information is then signed by a trusted authority using asymmetric encryption. The signed certificate ensures that the combination of public key and subject identity, for example, a domain name, has been reviewed and approved by the trusted authority. As formally defined in Appendix B.1, a X.509 certificate consists of information about the signature (Signature Algorithm and Signature Value) and the essential data needed to establish a connection, which is known as To Be Signed Certificate (TBSCertificate). Table 2.1 provides a description of the key attributes in a X.509 certificate.

2.1.3 PKI Hierarchy

As mentioned above, when creating a digital certificate, a trusted instance is needed to issue and sign the certificate for its validity. This trusted instance is called Certificate Authority (CA). A CA is a special type of public key certificate, where the associated key pair is used to issue certificates for other CAs or end-entities. The hierarchical signing of CAs leads to a certificate chain for each certificate. The top-level CA in this chain is called *a trust anchor* (also called root CA). All CAs between the trust anchor and the end entity

Certificate Type	Description
Self-issued	A certificate where issuer and subject are the same CA. Used to perform a key rollover or renew a CA certificate
Self-signed	A certificate which has been signed with the corresponding secret key. Used by trust anchors, starting the chain of trust. Those secret keys are secured in HSMs in combination with additional security measures
Cross-certificate	A certificate that was signed by different CA, used to instantiate new CAs or distribute the trust of a CA by using multiple signatures

Table 2.2: *Different types of CA certificates defined in ITU X.509 Clause 7.4 [27]*

certificate are called intermediate CAs. Table 2.2 describes the three different types of CA certificates that can be issued in relation to the situation.

When an entity interacts with a PKI for encryption or verification, it has to trust the whole concept and is called a relying party, since it has to rely on the security of PKI. In addition, as the name indicates, the trust anchor certificates must be recognized as trustworthy by a relying party. For this purpose, a relying party has to pre-populate their trust anchor store (abbrev. trust store), in which all certificates are listed that the relying party is trusting for validation. Figure 2.4 visualizes the correlation between different types of certificates and the resulting certificate chain.

2.1.4 Public Trust vs. Private Trust

Although it is called “Public Key Infrastructure”, the term “Public” only refers to the term “Public Key Cryptography”. In fact, it is common to establish a PKI ecosystem in a private environment. This requires maintaining and establishing their own trust anchor and ensuring that participating devices recognize those CAs. The usage of a private PKI is highly correlated with the extended-key-usages defined in the certificates. Use cases like VPN connections or the use of internal software do not need to be publicly available and, therefore, the whole PKI can be kept in a private network [17]. Implementations of PKIX that underpin the Web PKI form part of a public PKI, where the certificates are typically accessible over the Internet.

2.1.5 Certificate Revocation List and OCSP

Although X.509v3 certificates contain a validity period, a mechanism is needed to invalidate a certificate before the original time of invalidity, if the secret key or the issuing CA have been compromised. ITU X.509 [27] specifies Certificate Revocation List (CRL) as a basic approach by using signed lists to define which public keys are no longer valid. A CRL can only be issued by a CA and therefore can only cover the certificates issued by this CA. In addition, to invalidate public key certificates before their expiration, ITU X.509 defines various other types of list to update certificates retroactively, such as Certificate Authority Revocation List (CARL), Attribute Validation List (AVL), or Attribute Certificate Revocation List (ACRL).

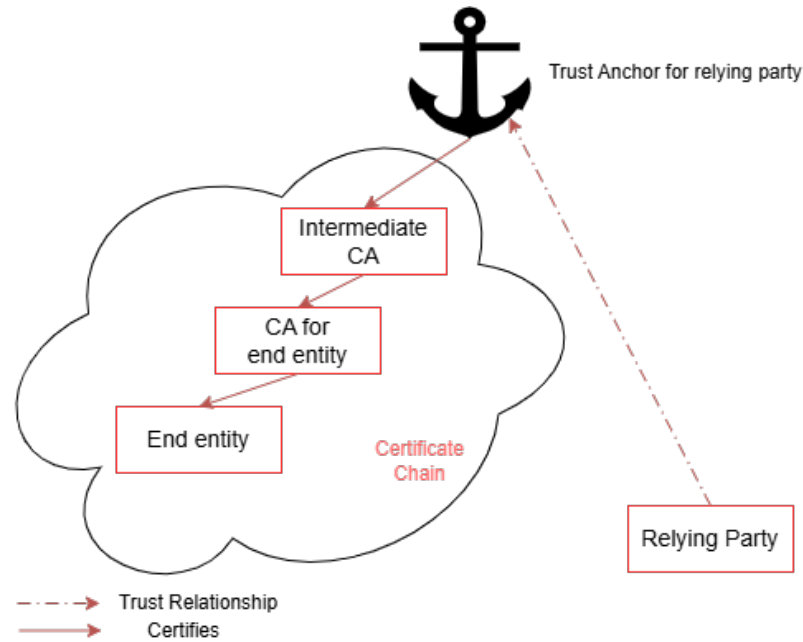


Figure 2.4: Visualization of trust relationships in a PKI. Adapted from ITU-X.509 [27]

An alternative approach to verifying the validity of a certificate is to use the Online Certificate Status Protocol (OCSP) instances defined in RFC 6960 [52], which then may use a CA’s CRL or another mechanism to determine the validity.

2.2 Public Key Cryptography Standard

The PKIX working group explicitly mentions, that their standards only suggest on how to use X.509 certificates in a PKI ecosystem. Those standards do not contain proposals on which cryptographic systems or schemes a concrete implementation should use.

However, the “RSA Security Inc.” addresses exactly these topics. The company in cooperation with developers worldwide published specifications for their ideas called Public Key Cryptography Standard (PKCS) [36]. The list contains 15 best practices for using various cryptographic systems and schemes. The current version of PKCS#1 is described in RFC 8017 [36], for example. It contains recommendations on how to use algorithms based on RSA, such as RSAES-PKCS1-v1_5 or RSASSA-PSS. Although it is not an actual standard, implementations around the world rely on these best practices.

2.3 Certificate Transparency

The DigiNotar compromise in 2011 expressed the need for an approach to monitor and audit certificates issued by CAs, since attackers were able to issue new certificates for common websites and provide a surface for MITM attacks [35]. Therefore, a concept

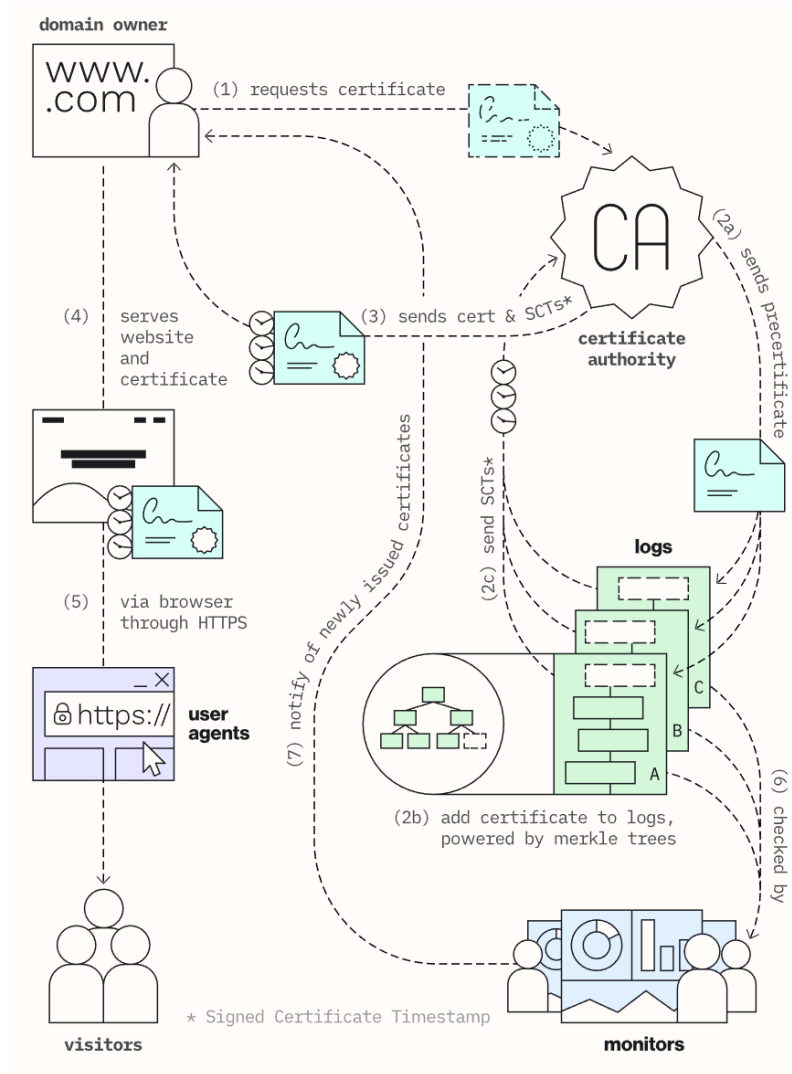


Figure 2.5: Integration of CT into Workflow of Issuing Certificates by [13]

was needed to recognize when certificates are issued without authorization from the domain owner. This approach is called CT and was initially developed and deployed by *Google* before being standardized in RFC 6962 [33]. This concept is primarily enforced on certificates issued in the Web-PKI, due to its dominance in PKIX and because Web Browsers, such as Chrome or Firefox, are the main drivers for this concept. Figure 2.5 visualizes which additional components are needed and how they interact with each other, to achieve a fully transparent workflow to issue new certificates.

RFC 6962 first defines CT logs as the base data structure. Those logs are public, append-only data structures that are cryptographically secured against tampering by leveraging Merkle trees. When a CA receives a request for a new certificate, it sends the certificate to one or more CT logs that return a Signed Certificate Timestamp (SCT). This timestamp is a guarantee by the log that it will insert this certificate into its log within a fixed amount

of time. The CA now returns the signed certificate along with the SCT back to the domain owner. Now when the domain owner has to authenticate itself, it has to provide its certificate together with the SCT, otherwise the client shall terminate the connection. Therefore, the client can use the SCT to look up the log entry and compare the signatures for verification.

However, only the log does not prevent the creation of misissued certificates. Two special clients are described in RFC 6962 that enable the desired transparency. *Monitors* continuously fetch the tree head of a CT log until the head signature changes. The updated head indicates that new entries have been added to the log, which are now monitored and verified by performing a consistency proof. If all entries are valid, a monitor can alert the corresponding domain owners if they ordered a subscription. Therefore, the domain owner would be able to detect misissued certificates when it receives a notification without requesting a new certificate.

2.4 Post-Quantum Cryptography

The digital world is evolving rapidly. However, the evolution of quantum computers not only solves problems, but also comes with new risks and threats. The HNDL threat is one such problem, as currently considered safe communication is archived and will be decrypted once CRQCs can be used efficiently to break this encryption using algorithms such as Shor's or Grover's [51].

In order to remain resilient against the era of quantum computers, NIST started the PQC Project, where post-quantum encryption algorithms can be submitted from which a subset will be used for standardization. NIST has now finalized three standards for PQC that are ready for immediate use [41]. Furthermore, additional algorithms, such as fast-Fourier transform over NTRU-Lattice-Based Digital Signature Algorithm (FN-DSA) or Hamming Quasi-Cyclic (HQC), are still pending for standardization in the upcoming years [1, 2]. However, the transition of PKI towards PQC is still long, since NIST set a deadline to 2035 to deprecate and remove quantum-vulnerable algorithms from their standards. This implies a roughly 10-year transition period for organizations [51].

2.4.1 PQC Algorithms

Conventional cryptography systems and schemes rely on mathematical problems, such as *prime factorization*, *discrete logarithm*, and *modulo operation*. However, quantum computers are capable of solving those problems using Shor's and Grover's algorithm [61]. Therefore, PQC algorithms must rely on different mathematical approaches to remain secure against CRQCs as well as conventional computers [51].

Lattice-based Cryptography

The first two standards FIPS 203 [40] and FIPS 204 [39] rely on the Module Learning With Errors (MLWE) problem, which is based on lattices. The FIPS 203 standard defines

Category	Exemplary Algorithms
Code-based	Classic McEliece, BIKE, HQC*
Isogeny-based	CSIDH
Multivariate	GeMSS, Rainbow

Table 2.3: *Additional categories in the field of PQC with exemplary algorithms that have been considered for standardization by NIST at some point [1]. Algorithms marked with * will be standardized in the near future [2].*

Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) in three variants $[-512, -768, -1024]$. It can replace the classical key establishment schemes defined in SP 800-56A [7] and SP 800-56B [8]. FIPS 204 proposes the signature scheme Module-Lattice-Based Digital Signature Standard (ML-DSA). It intends to replace the classical digital signature schemes defined in FIPS 186-5 [38]. This standard also introduces three variants ML-DSA- $[44, 65, 87]$

Hash-based Algorithms

Classical hash-based signature schemes, such as Leighton-Micali Signature (LMS), Hierarchical Signature System (HSS) or eXtended Merkle Signature Scheme (XMSS), are stateful and rely on hash functions such as SHA-256 or SHAKE-256 to generate their key pairs. The infeasibility to reconstruct the input of a certain hash function based on the output is considered secure against large-scale quantum computers, making hash-based algorithms viable for PQC. However, the state management for these algorithms is crucial, because obtaining two signatures using the same state enables the possibility of forging signatures. Therefore, stateful hash-based algorithms are not suitable for general use and instead are part of larger schemes [16].

FIPS 205 [42] now introduces Stateless Hash-Based Digital Signature Standard (SLH-DSA) as a backup approach for ML-DSA [41]. This algorithm is a combination of the multi-time signature scheme XMSS and the few-time signature scheme Forest of Random Subsets (FORS), resulting in a stateless signature scheme.

Other Categories

Although until now only lattice-based and hash-based algorithms have been standardized, there are more mathematical base problems that allow different algorithms to be secure against quantum computers [44]. Table 2.3 displays three other categories and exemplary algorithms.

2.4.2 PQC Integration Types

The creation of PQC algorithms is only the first part in making PKIs secure against quantum computers. Depending on the progress of the transition and the PQC readiness of relying parties a PKI can choose among several integration approaches [44, 60].

Quantum-safe Certificates

When all relying parties are able to support the PQC algorithms, the certificate can be issued using one of the new algorithms and be considered quantum-safe. This results in a classic X.509 certificate where all algorithm specific values have been exchanged with PQC components, as shown in Figure 2.6.

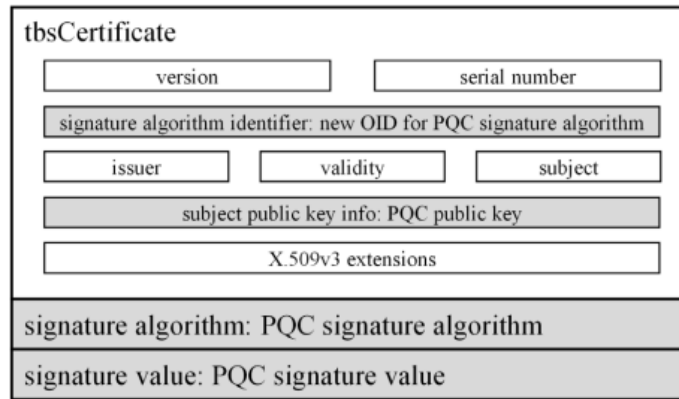


Figure 2.6: Schematic view of quantum-safe certificate by Congli Wang et al. [60]

Hybrid Certificates

A transitional solution is the usage of the “subject alternative signature” extension defined in International Telecommunications Union (ITU) X.509 clause 9.8 [27]. In doing so, the TBSCertificate is signed using traditional algorithms and a PQC algorithm and provides both signatures to the relying party. Once all relying parties have transitioned to PQC, hybrid certificates can be replaced with quantum-safe certificates. Figure 2.7 shows which extensions are needed to be fully compliant with the concept of hybrid certificates.

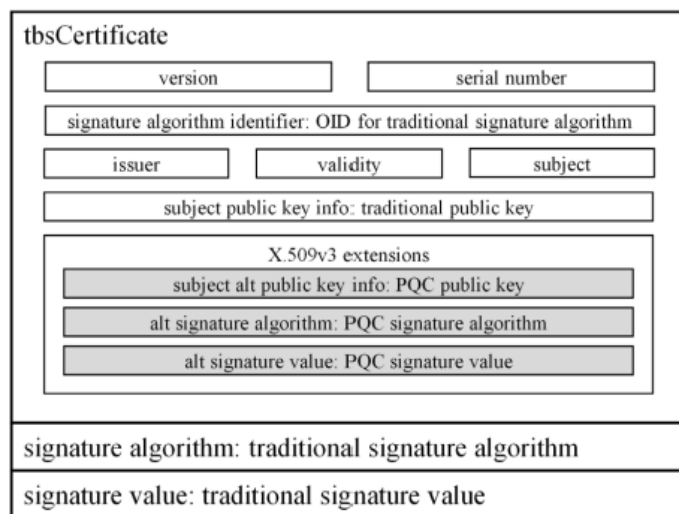


Figure 2.7: Schematic view of hybrid certificate by Congli Wang et al. [60]

Composite Certificates

Another approach is to combine traditional algorithms and PQC algorithms by applying multiple algorithms in sequence. By concatenating the keys of each algorithm a single key or signature value is generated. However, this also requires that all relying parties are already capable of handling both PQC and traditional algorithms. Figure 2.8 shows how the concatenation can be applied.

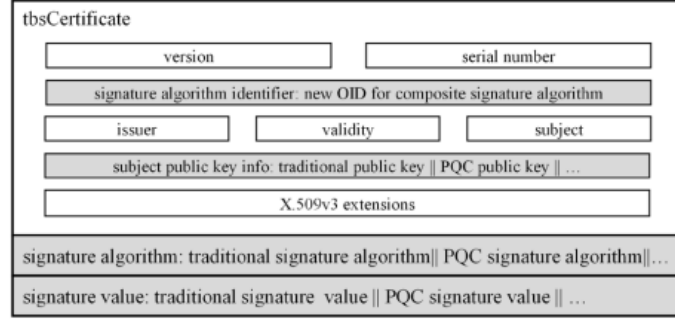


Figure 2.8: Schematic view of composite certificate by Congli Wang et al. [60]

Parallel Certificates

This approach uses two parallel certificate chains, where one chain uses traditional algorithms and the other chain uses PQC algorithms. So, for each entity in the traditional chain, a corresponding certificate exists in the PQC chain. When the transition is complete, the traditional chain can be deprecated, and only the new chain acts as a normal chain with quantum-safe certificates.

Mixed Certificate Chains

A common problem in PKIs is that a dedicated certificate chain mostly relies on a single cryptographic system. Therefore, the relying parties only have to be capable of handling this explicit type of cryptography. Sebastian Paul et al. [44] propose the usage of mixed certificate chains to realize the transition towards quantum-safe PKI implementations. This approach first applies well-studied hash-based signatures at the root level while keeping conventional signature schemes at lower levels. It is intended to ease the transition, especially given the long lifetimes of root CA certificates. In a second step, the lower levels are updated with lattice-based signatures to provide full post-quantum authentication.

2.5 Column-oriented Databases

Besides traditional row-oriented database systems like MySQL, there are column-oriented databases, such as *Clickhouse* or *Apache Doris* [4]. The naming indicates that they differentiate by how the data is stored in the actual file system.

Although column-oriented databases still see a new entry as a single row, individual attributes are split into different files. In doing so, each attribute ends up in a separate file dedicated to the specific column. When an empirical analysis is performed, the data sets can grow to large dimensions. However, that is when a column-oriented approach can significantly speed up aggregations. First of all, since the column data is separated from each other, the database is able to evaluate a certain subset of columns without worrying about the other columns. This concept comes in especially handy when performing aggregations over a certain number of columns. However, every strength comes with trade-offs. Although column-oriented databases excel at analyzing data, they are less efficient when modifying data. Whenever rows are inserted, updated, or deleted, the database must access multiple files instead of just one. [14]

3 Related Work

The progress of current research in the field of empirical analysis for PKI will be evaluated in various categories. Mostly, different empirical studies, such as incident studies or general PKI analyses, will be examined. Furthermore, articles dealing with the basis are included, such as measurement of PKIX or new approaches in the field of PQC.

3.1 Measurement of PKIX

Data collection and measurement are crucial for empirical analysis. In 2015, Zakir Durumeric et al. [18] introduced *Censys*, a public search engine, providing various information about the hosts and services running in the public IPv4 address space, including information about the X.509 certificates used. Although *Censys* provides various options to interact with the data, such as aggregation and search queries, the search engine is limited to single requests, and combining multiple results may not always be possible. Therefore, to gain more complex results on the same data set, their API or raw data needs to be integrated into different tools, which specialize on the evaluation and do not have to be dealing with the data aggregation. Pouyan Fotouhi Tehrani et al. [58] provide a tutorial on the different aspects of collecting data and measuring the Web PKI. They list a set of measurement features and distinguish between snapshot and longitudinal measurement frequency. Furthermore, they present various types of data sources such as Hit Lists, CT Logs, or Indexing Services (e.g., *Censys* or *Rapid7 Open Data*). In addition to giving concrete instructions on how to perform an extensive analysis, this paper does not actually perform any analysis with further goals. Benjamin Vandersloot et al. [59] also dealt with the topic of data collection in PKIX. They also compared various data sources, similar to that Pouyan Fotouhi Tehrani et al. did, but aimed to provide the best and most complete view of the PKIX ecosystem. They propose combining the *Censys* index service with the monitoring of CT Logs to achieve a coverage of about 99.4%.

Aozhuo Sun et al. [55] conducted a 52-day trial analyzing 6 popular active third-party monitors for CT, highlighting the advantages and disadvantages of each service and providing a detailed comparison for selecting an appropriate provider. In doing so, they evaluated specific metrics like service delay, but did not contribute to the analysis of PKIX and its characteristics.

3.2 Incident Studies on PKIX

Although the concept of PKIX aims to improve Internet security, there are several studies on incidents in this ecosystem. Salabat Khan et al. [30] conducted a comprehensive survey about PKIX comparing various proposals to improve PKIX. They cluster different proposals by which class is able to solve a certain problem and furthermore list their strengths and weaknesses, allowing them to make suggestions on which proposals are

more promising. However, they only compare on a theoretical level in the form of a literature review and do not test the approaches empirically.

Hilda Hadan et al. [21] interviewed experts in the field of PKI, asking for failure modes of PKI and possible solutions. Later, they compared the results with results from a metadata analysis on real-world PKI incidents in the period of time from 2001 to 2021, based on data from various sources such as Bugzilla, security reports, or academic literature. They concluded that expert views are contrary to their findings, as incident analysis shows that a major cause of PKI failures originates from non-compliant CAs in combination with slow reactions to compromised certificates and CAs. However, experts mostly identified missing or expired certificates as the root problems. Multiple studies have been conducted on how different threat models, such as HTTPS-Phishing [31] or Certified Malware [32], are enabled through bad practices of CAs or relying parties. In doing so, they analyze specific subsets of PKIX, such as lists of known phishing websites or telemetry metrics from real-world firmware images.

3.3 Certificate Transparency Honeypot Analysis

The introduction of CT Transparency in 2011 led to increased trust in PKIX. However, enforcing public logging enables new attack vectors. Studies by Quirin Scheitle et al. [53] or Stijin Pletinckx et al. [47] use honeypot websites to track incoming traffic after issuing a new certificate to a CT log. The almost immediate network traffic may not only indicate classic monitoring of those logs but also indicates malicious monitoring used for target reconnaissance.

3.4 IoT and Constrained Environments

Mohammed El-Hajj and Pim Beune [22] performed an extensive review of the literature on how to efficiently apply the concepts of PKI to the Internet of Things (IoT). The approaches range from optimized X.509 certificates for full compatibility with PKIX or only leveraging ECC and the resulting small key pairs.

Joel Höglund [24] wrote his dissertation on how X.509 certificates can be used in resource-constrained IoT environments using Concise Binary Object Representation (CBOR) encodings.

3.5 Alternatives to PKIX

In 2014 Ralph-Günther Holz [25] conducted an empirical analysis as part of his doctoral thesis. He compared the status quo of three different approaches towards PKI: X.509, OpenPGP, and SSH's simple key distribution. His analysis revealed that the X.509 certification lacks in stringency. In addition to the empirical analysis, Holz analyzed the criticism and incidents of PKIX and lastly provides five schemes to improve the security of PKIX, which tackle some problems but also lack automatic incident reporting and localization of the position of an attacker. The analysis of PKIX used active scans using hosts around the

world in combination with lists such as Alexa Top 1 Million, providing a comprehensive data source. He also looked at similar characteristics, such as the distribution of signature algorithms, to provide a representative overview. However, the results depict the state of the art from 2014 and the concept was not laid out to be used repeatedly, therefore not being usable today.

3.6 Analysis of PKIX

Gurur Öndrö et al. [43] analyzed 41 million S/MIME certificates according to secure algorithms and regulations such as the CA/Browser's Forums' S/MIME Baseline Requirements. They found that S/MIME PKIX is moving in the right direction, as newly issued certificates are more secure than already old and expired certificates. Stefan Tatschner et al. [57] used *Censys*'s certificate collection to compare the output and behavior of 6 different parsers, finding large differences depending on the chosen parser. In addition to their findings, the authors published the code used to collect and analyze the data [56], which provides a solid foundation on how to download raw data from *Censys*. However, adjustments are needed when the data format is altered or the data shall be stored in a database instead directly on the file system as JSON. Syed Muhammad Farhan and Taejoong Chung [20] performed an empirical analysis observing the Google CT log and certificates from complete IPv4 scans from 2013 to 2021. They tracked the evolution in this period of time looking at validity in combination with CRL and the signature algorithms that are most widely used. Although this paper observed the transition from SHA-1 signatures towards SHA-256 signatures, it was dated way before the standardization of PQC algorithms. Furthermore, this paper does not provide a concept for performing a newer analysis without executing the described procedure again. Anuar Magliari [34] proposed DigitalCertiAnalytics. A tool used to collect, verify, and analyze X.509 certificates from a desired set of domains. The tool allows to analyze every common attribute of an X.509 certificate, which was then compared to the baseline requirements of the CA/B Forum. Although the tools allows one to analyze an arbitrary number of certificates, the thesis used two sets of each 10 million domains and, therefore, only depicted a fraction of the globally used PKIX. In addition, this tool would be resource-demanding when executed regularly, since the domains have to be scanned again. Regarding the evolution of PQC algorithms, the tool does not support the analysis of alternative signatures in X.509 certificates, which may be essential in the transition from classical cryptography to PQC.

3.7 Post-Quantum Cryptography

The evolution and progression of PQC is essential to ensure the continued security in PKIs even in the era of quantum computing. Manohar Raavi et al. [49] compared the different algorithms in terms of feasibility and memory performance, concluding different algorithm recommendations based on application requirements. Corey Bonnell et al. [12] propose an alternative approach to transition towards PQC algorithms using the so called chameleon certificates. In doing so, a conventional base certificate is enhanced with a

descriptor pointing to a delta certificate. The delta certificate allows one subject to possess multiple keys by defining a second key pair. This approach is similar to hybrid certificates and can be used to introduce another key pair using a PQC algorithm. Lastly, the Public Key Infrastructure Consortium (PKI Consortium) maintains an overview of PQC adoption called PQC Capabilities Matrix (PQCCM) [45]. The matrix contains most of the global players of the PKI ecosystem and which PQC concepts are supported by the particular instance.

3.8 Summary

Although PKIX is a critical part of today's communication security, only a few empirical studies have analyzed PKIX on a global scale. Furthermore, it often is only a time-dependent snapshot and rarely a setup to regularly perform new analyses.

This thesis advances prior work by: (1) implementing a reusable analysis platform (*PKI Explorer*), enabling periodic evaluation, (2) analyzing as many certificates as possible without limitations to filtered lists, and (3) one of the first empirical measurements of PQC adoption in productive ecosystems.

4 Concept

This chapter describes the analytical approach and methodology for this thesis. First, the analysis strategy is presented with the goal of answering the research questions. The second section provides an introduction to the possible data sources for the empirical analysis.

4.1 Empirical Analysis

To answer the research questions in this thesis, an empirical analysis of the PKIX ecosystem will be conducted. This analysis will track the distribution of X.509v3 certificate attributes over time by performing monthly calculations on regularly updated data. Furthermore, to enable a temporal comparison of how the ecosystem evolves, the analysis results will be timestamped and stored in a database.

In this thesis, the following attributes will be analyzed, providing information on current changes in PKIX. The general tooling for this thesis will be designed to be extendable to other attributes in the future.

4.1.1 Certificate Usage Analysis

When analyzing a PKIX ecosystem, one of the most relevant metrics is determining which certificate use cases are represented in the ecosystem and identifying the dominant ones. X.509v3 certificates provide two mechanisms to specify usage: (1) coarse-grained *Key Usage* and (2) fine-grained *Extended Key Usage*. Both will be analyzed to understand the distribution of different certificate purposes in PKIX.

4.1.2 Algorithm Analysis

The transition towards quantum-safe PKIX ecosystems can be tracked by inspecting the signature and encryption algorithms used. Therefore, all possible entries for cryptographic algorithms will be analyzed to get an understanding of the transition towards PQC algorithms.

Since these fields use OIDs to specify the algorithm used, and PQC standards are continuously being finalized, a curated registry of PQC algorithm OIDs will be maintained (see Table 4.1).

Standard Signature and Key Algorithm

A X.509v3 certificate typically contains two cryptographic algorithms: (3) Subject Public Key Algorithm and (4) Signature Algorithm.

Algorithm Name	OID
ML-DSA-XXX	2.16.840.1.101.3.4.3.[17-19]
ML-DSA-XXX-ipd	1.3.6.1.4.1.2.267.12.*
SLH-DSA-XXX-XXX	2.16.840.1.101.3.4.3.[20-31]
SLH-DSA-XXX-XXX-ipd	1.3.9999.6.*
HASH-ML-DSA-XX (Pre-Hash: XXX)	2.16.840.1.101.3.4.3.[32-34]
HASH-SLH-DSA-XXX-XXX (Pre-Hash: XXX)	2.16.840.1.101.3.4.3.[35-46]
ML-KEM-XXX	2.16.840.1.101.3.4.4.[1-3]
ML-KEM-XXX-ipd	1.3.6.1.4.1.22554.*
Composite ML-DSA	1.3.6.1.5.5.7.6.[37-54]
Dilithium-XXX	1.3.6.1.4.1.2.267.7.*

Table 4.1: *Reduced overview of PQC algorithms and the corresponding OID ranges. Appendix C contains lists for complete overview.*

Hybrid Certificates

The ITU X.509 [27] defined a set of extensions to enable smooth transitions between different cryptographic algorithms by providing two sets of public keys and signature values. Although these extensions are not that popular, they may come in handy for the transition towards PQC algorithms. Especially, since it will be common that only a subset of all relying parties will support PQC algorithms. Therefore, the corresponding extensions will also be analyzed: (5) Alternative Public Key Algorithm and (6) Alternative Signature Algorithm.

This concept for algorithm analysis can determine the amount of certificates that have already started to transition towards PQC algorithms or are already fully quantum-safe. Although this concept can differentiate between quantum safe certificates and hybrid certificates, it will not be able to distinguish quantum safe certificates from parallel certificate chains, since this would require additional analysis. Consequently, this thesis can detect quantum-safe or hybrid certificates at the individual-certificate level but cannot systematically distinguish parallel certificate chains without additional domain specific correlation logic.

4.1.3 Metadata Filter

When inspecting a certificate used in PKIX, the attached attributes allow one to cluster certificates on a meta level. Thus, distribution charts can be generated for the coarse subset of the PKIX.

Certificate Type

The hierarchal differences between the trust anchors, intermediate CAs, and end-entity certificates are crucial in terms of the default values. In particular, the number of certificates dedicated to a certain level differs enormously between trust anchors and end entity certificates. Furthermore, the main purpose of a certificate differs between CAs and end entity certificates. Therefore, certificates will be distinguished between the states:

- Root
- Intermediate
- Leaf
- Unknown (fallback)

Trust status

Within the set of certificates found on the public Internet, the validity status of a certificate is a strong indicator of whether it is used in production. The validity of a certificate can be checked with tools like Zlint and the simple check for the validity period. The distinction between invalidity is important due to expiration or due to non-compliance to CA/B Forum baseline requirements. Therefore, it will be distinguished between the states:

- trusted
- was-trusted
- untrusted (fallback)

CT Relation

Another indication of a valid certificate is the proper logging of the certificate in a CT-log. *Censys* monitors several CT logs, such as *Google*, *Let's Encrypt*, or *Comodo*¹. Although *Censys* differentiates between multiple logs of the same vendor, this analysis will only use the vendor generally.

Locality

To get a grasp on which localities are tracked around the world, the subjects country value will be evaluated. RFC 5280 [11] defines that a DN for a subject shall use ISO 31-66-1 alpha-2 country codes.

4.2 Data Source

The concept of PKIX is implemented in various systems around the world. To obtain an almost complete view of PKIX, various observation points will be evaluated, whereas *Censys* provides the largest data set.

4.2.1 Indexing Services

The scope of PKIX is constantly growing and scraping this volume of data independently becomes more challenging over time. Platforms like *Censys*² or *crt.sh*³ specialized in collecting as much information about the web space as possible by performing IPv4-Scans

¹List of CT logs observed by *Censys*: <https://docs.censys.com/docs/ls-certificates#certificate-collection>

²<https://censys.com/>

³<https://crt.sh/>

or monitoring CT logs [58]. B. VanderSloot et al. [59] conducted a study on how to achieve an almost complete view of the PKI ecosystem. The result is that services such as *Censys* cover a large part of the ecosystem. During their studies, *Censys* did not support the monitoring of CT-logs, leaving a gap. However, in recent years, *Censys* has expanded their services by also providing such monitoring. Therefore, this thesis will use the data provided by *Censys* to cover the public PKIs.

Censys supports research activities by providing three types of access to their data set for research purposes. The different types will be described and evaluated, which is best suited for this type of empirical research.

Search API

Censys operates a legacy search engine, which is still maintained, although they developed a new platform for industrial purposes. In addition to using their web interface to perform search queries against their data set, *Censys* also provides access to an API for their search engine.

However, even though a specific endpoint for aggregation is available, not all needs can be satisfied, since the parsed data-structure is the limiting factor. Especially a set based aggregation of “Extended Key Usage” is not possible, since those entries are stored in a large structure of multiple boolean instead of a set containing the found values. Furthermore, *Censys* limits the number of requests to 25.000 queries per month. Considering the exponential amount of key usage combinations and adding the combinations for meta filters, this limit is reached fast.

Although the search API does not seem to be suitable for empirical analysis, it proved to be helpful during the concept phase. The possibility of searching for specific certificates and inspecting the parsed object, the data points needed for the empirical analysis, was identified.

Google BigQuery Repository

Censys also provides read-only access to their underlying *Google BigQuery* repository. The costs of performing queries against this repository have to be borne by the researcher.

Furthermore, the repository has similar limitations regarding the data structure of extended key usages. Although the set aggregation may be mitigated by using complex temporary tables, the parsed version of the “Alternative Signature Extension” is still missing. Therefore, highly complex queries would be needed to extract the desired information. In combination with the rather unpredictable cost-model of *Google BigQuery*, a long-term analysis of the data set is not cost-efficient.

Data Download

The third option provided by *Censys* is a complete download of their data set. The data replica then has to be stored on one’s own hardware. This option allows one to alter the data model to their needs via prior processing. However, to solve the data model problem,

the costs of managing the hardware for data storage will be added. This factor also brings new risks and requires extensive planning, considering the size of the data source.

After evaluating all options, this thesis uses the raw data download for data access. This enables a custom data model that supports extended key usage analysis. Furthermore, the cost model is more predictable than with *Google BigQuery*. Especially since using a self managed database enables extensive experimental trials with low additional costs.

4.2.2 Firmware Images

Since indexing services are mainly relying on address scanning and public logs, many areas of PKIX remain undetected. Especially, the field of IoT is growing similarly fast. However, due to limited available resources, IoT devices often predefine certificate-related information in their firmware. Since these firmware images cannot be queried or scanned, a set of public collections of firmware images will be used. The selection of lists determines which other sectors of PKIX will be inspected. Some lists can be generic, like the list provided by Yuhao Wu et al. [62], which covers various vendors and entries from public FTP endpoints. However, there are specific lists, such as the data provided by Grant Hernandez et al. [23], which contains firmware images for cellular baseband communication in mobile phones.

Unlike the services provided by *Censys*, the lists only provide the firmware images in binary format. Therefore, before further processing, PKIX-relevant certificates have to be extracted. For this purpose, tools such as Binwalk⁴ can be used. Those can scan a binary file for common file signatures, particularly for certificates.

Due to time and storage constraints, this data source was not implemented in this thesis and remains conceptual future work.

4.2.3 Data Definition

In order to evaluate the various attributes of the gathered X.509 certificates, they need to be stored in a parsed data format. Since the firmware images only contain the binary value of certificates and *Censys* provides an already parsed data format, this data structure will be used for the internal representation.

In addition to parsing the attributes of the X.509 certificates, *Censys* enhances their data definition by providing service oriented metadata, such as insertion or modification timestamps, and introduces tags, allowing simplified clustering, such as differentiation between root, intermediate, and leaf certificates⁵. Furthermore, it sometimes occurs that *Censys* has trouble parsing certain certificates due to unknown reasons. Such failures are indicated using an additional attribute called `parse_status`. Possible values for the status are `CERTIFICATE_PARSE_STATUS_SUCCESS` or `CERTIFICATE_PARSE_STATUS_CORRUPTED` or `CERTIFICATE_PARSE_STATUS_FAILURE`.

Therefore, the certificates found in the firmware images need to be parsed into the same data structure to ensure comparability between both data sources.

⁴Binwalk Repository: <https://github.com/ReFirmLabs/binwalk>

⁵Data definition by *Censys*: <https://search.censys.io/search/definitions?resource=certificates>

5 Implementation

This chapter describes how the analysis strategy is applied to the selected data sets. It starts by laying out the intended architecture for *PKI Explorer*. The following sections describe the implementation of the particular steps. The chapter ends with a summary of operational troubles during the implementation and execution of the analysis.

5.1 Architecture

The management and evaluation of large data sets require prior architecture planning to minimize the risk of inefficient and error-prone processes. First, Table 5.1 shows the specification of all hardware units available that will be used to realize the platform for this thesis. Furthermore, the university offers the option to set up a virtual Kubernetes cluster¹ that is integrated into a real cluster, offering additional dynamic resources on demand.

Figure 5.1 displays which subsystems are planned, how they are distributed, which technologies will be used, and how they interact with each other. It also shows that the realized implementation does not include the processing of firmware images, since the time and disk space ran out after the implementation for the *Censys* data.

5.2 Data Storing

In an empirical analysis, the selection of the data management system is crucial. Since the goal focuses on distribution statistics, a column-oriented DBMS was chosen that excels in performing aggregations on columns and managing large data sets. Popular column-oriented database systems include *Clickhouse*², *Apache Doris*³, or *DuckDB*⁴. This implementation uses *Clickhouse* mainly because of the native support of dynamic JSON data structures and the simple setup process.

¹Kubernetes Cluster maintained by University of Applied Sciences Munich: <https://kube.cs.hm.edu/>

²<https://clickhouse.com>

³<https://doris.apache.org/>

⁴<https://duckdb.org/>

	HP DL370 G6	Supermicro X9SCL/X9SCM
OS	Ubuntu	Ubuntu
CPU	2x Intel Xeon X5650	Intel Xeon E3-1220 v2
RAM	96GB	16GB
Disk Space	14x 3TB	15x 4TB
Additional Information	HW RAID Controller	HW RAID Controller

Table 5.1: Available hardware units, provided by the University of Applied Sciences Munich

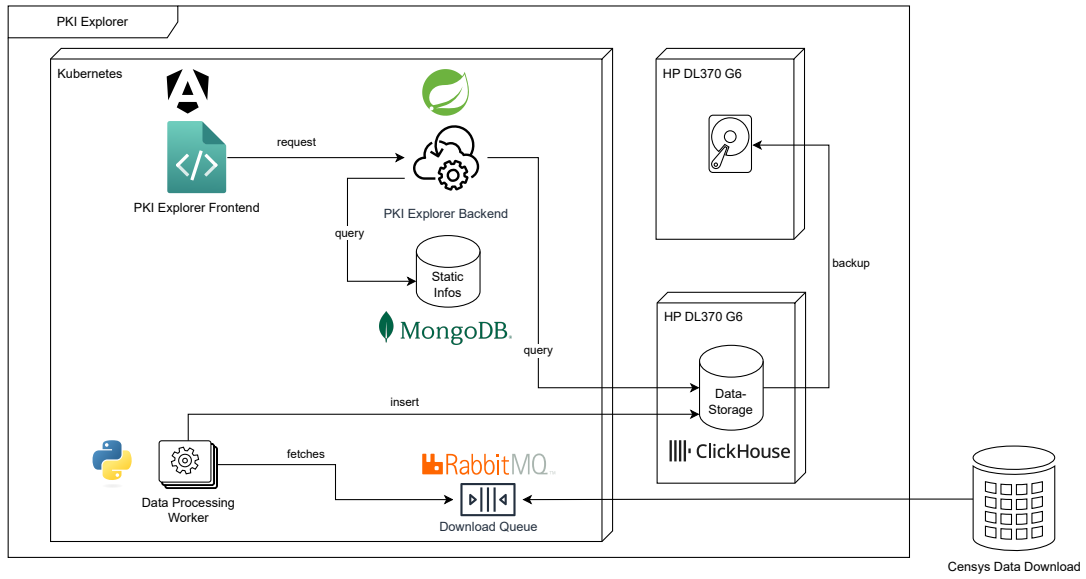


Figure 5.1: High-level architecture of PKI Explorer

A *MongoDB* instance is deployed additionally in the Kubernetes cluster. This database is used by *PKI Explorer* to store additional information. For this implementation it only contains the mapping from OIDs to clear names.

Clickhouse is an open-source DBMS that belongs to the subcategory of online analytical processing (OLAP) databases. Therefore, their focus is on processing large amounts of data in real time.

5.2.1 Installation

Clickhouse offers various installation types ranging from cloud integrations over docker images towards native installations on various operating systems. This implementation followed the 4-step guide⁵ needed to set up an instance on an Ubuntu system. In addition to starting the service, this installation provides a CLI client to directly interact with the database for initial configuration and schema creation.

After initial boot, two configurations have been adjusted. First, the database needs to be accessible from the outside. Therefore, the *listen_hosts* value was updated to “::”, accepting connections from everywhere. Second, since the HP server has 24 virtual cores, the amount of *background_pool_size* for internal data management was increased to 20, resulting in 40 possible parallel background tasks.

⁵*Clickhouse* installation guide for Ubuntu systems: https://clickhouse.com/docs/install/debian_ubuntu

5.2.2 Data Schema

For the analysis, multiple tables are needed: (1) certificates, (2) pki_analysis_results, (3) alt_public_key_analysis, and (4) pki_analysis_corrupt.

The certificates table will be filled with the initial data set and is then updated regularly with the incremental data sets. However, the concept of *Clickhouse* does not support the classical update features that allow the existence of two entries with identical primary key. The corresponding alternative is the usage of the *ReplacingMergeTree* engine. This type of table will distinguish between multiple entries with the same primary key based on additional information or the values stated in the order by field. For this case, *Censys* provides the ideal attribute “modified_at”, allowing the replacing statement to keep the entry with the newest timestamp. The listing D.1 shows the top level structure of the final table.

The table for the main analysis results will be storing as many attributes as possible, including metadata. Therefore, enabling cross referencing between various values. Furthermore, to enable date based distinction between different results, another column is added containing the date of the analysis. Listing 5.1 shows the final definition of the table.

```
1 CREATE TABLE IF NOT EXISTS pki_analysis(  
2     extended_key_usage_set Array(String),  
3     key_usage_set Array(String),  
4     public_key_algorithm Tuple(  
5         oid String,  
6         keysize Int64,  
7         additional_info String  
8     ),  
9     signature_algorithm String,  
10    alt_signature_algorithm String,  
11    alt_public_key String,  
12    subject_country String,  
13    ct_entry_log String,  
14    cert_type Enum('leaf', 'intermediate', 'root', 'unknown'),  
15    trusted Enum('trusted', 'was-trusted', 'untrusted'),  
16    distribution Int64,  
17    query_date Datetime64) PRIMARY KEY(extended_key_usage_set,  
    key_usage_set, public_key_algorithm, signature_algorithm,  
    alt_signature_algorithm, alt_public_key ,  
    subject_country, cert_type, trusted, ct_entry_log,  
    query_date)
```

Listing 5.1: SQL statement used to create a table for persisting the analysis results.

However, since alternative public key information was not part of the preprocessing, they have to be analyzed separately. For this purpose, another table with the same metadata filters is needed, but different attributes for analysis.

Finally, since certificates that cannot be parsed do not contain any certificate relevant data, only metadata will be analyzed. Listing 5.2 shows the definition of the table to store the analysis results for those certificates.

```
1 CREATE TABLE IF NOT EXISTS pki_analysis_corrupt(  
2     ct_entry_log String,  
3     insert_year DateTime64,  
4     insert_quarter DateTime64,  
5     parse_status String,  
6     distribution Int64,  
7     query_date Datetime64) PRIMARY KEY(ct_entry_log, insert_year  
    , insert_quarter, parse_status, query_date)
```

Listing 5.2: SQL statement used to create a table for persisting the analysis results for certificates that cannot be parsed.

5.2.3 Tools and Integration

Clickhouse offers various integration options for the most common languages. For this implementation, the Java⁶ and Python⁷ libraries will be used. They include simple configuration for a client instance and additionally offer simplified functionalities that are wrapping SQL statements.

In addition to connecting to the database, the creation of backups is important to prevent possible data loss. *Clickhouse* itself provides the possibility of freezing a table or partition, creating static hardlinks as copies of the underlying data structure. Those hardlinks, in combination with metadata information, can be exported to the backup server. *Altinity* offers a tool called *clickhouse-backup*⁸ that leverages this functionality and offers various backup modes. The tool supports basic transfer protocols, such as FTP or SFTP. However, they also support custom configurations. Therefore, the backup implementation for this thesis uses a custom configuration, so data is transferred using *rsync*. This decision was made primarily because of the high error tolerance of *rsync*. Furthermore, *Altinity* provides an example on how to use *rsync* in a custom configuration⁹ and was used as a basis for the backup in this thesis.

5.3 Data Processing

The data download provided by *Censys* is accessible via their REST-API and is split into two series¹⁰. First, there is the initial download called “certificates-v2-full” and the followup series called “certificates-v2-incremental”. A single series contains several entries

⁶Java library for *Clickhouse*: <https://clickhouse.com/docs/integrations/java>

⁷Python library for *Clickhouse*: <https://clickhouse.com/docs/integrations/python>

⁸Backup tool for *Clickhouse* databases: <https://github.com/Altinity/clickhouse-backup>

⁹Example on how to use custom configurations for *clickhouse-backup*: <https://github.com/Altinity/clickhouse-backup/blob/47b62cdabb9507eeff3c176d3ea2881acd8b12e2/test/integration/config-custom-rsync.yml>

¹⁰Introduction on how to download the *Censys* data set: <https://docs.censys.com/docs/ls-download-certs-2-data>

distinguished by a timestamp. This allows the user to determine which data sets have already been downloaded. This is especially important when processing incremental updates. Querying a timestamp-specific series contains the download links for the compressed certificate data, using AVRO files. A single file then contains between 50.000 and 200.000 certificates.

This implementation used the full data set from June 01, 2025. This series contains around 220.000 files. Therefore, after processing the initial data set, this implementation would have continued with the incremental series starting June 01, 2025. However, due to a shortage in disk space on the data storage server, the initial data set was barely finished, leaving no room for incremental updates. It can be assumed that the order of the provided files is chronological. Therefore, the processed data set probably misses the last weeks until June 01, 2025.

To mitigate the risk of losing the progress of the processing, *PKI Explorer* uses a queue. Therefore, the actual processing is decoupled from the download management. A simple *RabbitMQ* queueing system was installed in the Kubernetes cluster¹¹. A small Python script then inserted all download links into a queue. In addition to decoupling the processing step from download management, the queue can be used to easily distribute the workload to a sufficient number of workers.

5.3.1 Worker Implementation

A single worker instance starts by connecting to the *RabbitMQ* queue waiting for messages. Upon receiving a response, the AVRO file has to be downloaded. However, due to initial testing or database related cyclic errors the download request can lead to a “TooManyRequests” error. *Censys* counts the download attempts executed by an IP address and blocks the download for 24 hours if it exceeds a certain threshold. In this case, an additional queue was created to store failed downloads. Those entries can be moved to the original queue once enough time has passed. This extra queue is needed, since the default behavior of the main queue is to re-insert failed messages in the front of the queue. This leads to blocked workers constantly failing due to failed downloads.

After the AVRO file has been successfully downloaded, the content is parsed into a dictionary using the *fastavro* library. From then on, the actual data modification starts. First, unnecessary attributes are removed, such as the raw certificate value. This reduces the size of the final certificates. Next, some data types must be converted to data types supported by the *Clickhouse* database. This applies to binary values and date times. Binary values are parsed into their hex representation. The date values are converted to a uniform format. To solve the original problem of set aggregations for (extended) key usage values, the boolean enumeration is inspected. Every entry with a true value is added to a corresponding set, which is then sorted to ensure uniqueness for later analysis. For the evaluation of hybrid certificates, the alternative algorithm extensions have to be extracted. Since *Censys* does not recognize them as default extensions, the binary values can be found in the “unknown extensions” attribute. During the initial implementation,

¹¹*RabbitMQ* installation guide for Kubernetes: <https://www.rabbitmq.com/kubernetes/operator/quick-start-operator>

it was first experimented with the alternative signature algorithms. However, later on the logic was never extended to also cover alternative public key algorithms, leading to increased effort for later analysis. Lastly, to ease the subsequent analysis, further entries are extracted into dedicated columns such as “CT Hit” or “Trust Status”.

The modified certificate batch is lastly inserted into the database. To mitigate inhomogeneity within a data batch, it is parsed into a Pandas data frame. Not represented values are replaced with 'None', which then leads to a null value inside the database. The python library provided by *Clickhouse* supports various python types for insertion, as well as Pandas data frames. Lastly, to reduce the load on the database, the python client uses asynchronous insert settings. Thus, the batch is not directly inserted into the table. The database instance buffers the inserted values in-memory and flushes it after a small delay or in case of a full memory.

5.3.2 Monitoring

The data provided by *Censys* had a size of around 15TB in summer 2023. Therefore, data modification and insertion into a database is a long running process. To track progress, various information was taken into account. First, the RabbitMQ operator deployed in the Kubernetes cluster comes with a dashboard overview. This visualizes various queue related statistics, such as amount of remaining messages and average acknowledgments per second. In addition to progress in replication, the database system should also be monitored. For this purpose, database metrics such as active data parts, ongoing merge processes, or simply the size of the table were tracked. Furthermore, basic information about the underlying operating system were regularly checked, such as resource usage or remaining free disk space.

5.4 Data Evaluation

The actual evaluation can be performed efficiently due to the column-based database systems. The listing 5.3 shows the final SQL statement that was used to execute the main analysis. To allow for cross references, every attribute to be analyzed is added to the group by statement. However, some attributes still need minor adjustments since they were not part of the preprocessing. For example, to extract the main vendor from the CT entry structure, a combination of string and array operations has to be applied. As a final column, the current timestamp is added to the result, which enables one to query different points in a timeline for later comparison. Furthermore, the table selection is appended with the keyword *FINAL*. This ensures that the replacement condition is met. Although no updates have been inserted, *Clickhouse* does store some values redundantly to cope with high input. Lastly, the certificates are filtered on the basis of their parse status. The subset of corrupted certificates must be omitted to avoid distorted distribution graphs.

```
1 INSERT INTO pki_analysis
2 SELECT extended_key_usage_set AS extended_key_usage_set,
3        key_usage_set          AS key_usage_set,
4        tuple( parsed.subject_key_info.key_algorithm.oid,
5              parsed.subject_key_info.rsa.length
6              parsed.subject_key_info.ecdsa.curve
7              ) AS public_key_algorithm,
8        parsed.signature.signature_algorithm.oid AS signature_algorithm,
9        alt_signature_algorithm AS alt_signature_algorithm,
10       arrayFirst(tuple -> tuple.id = '2.5.29.72', parsed.
11                 unknown_extensions).value AS alt_public_key,
12       upper(arrayFirst(entry -> (length(entry) = 2), parsed.subject.
13                 country)) AS subject_country,
14       arrayFirst(x -> true, splitByChar('_', arrayFirst(x -> true, ct.
15                 entries.key))) AS ct_entry_log,
16       certificate_type          AS cert_type,
17       trust_status              AS trusted,
18       COUNT(*)                  AS distribution,
19       today()                   AS query_date
20 FROM certificates FINAL
21 WHERE parse_status = 'CERTIFICATE_PARSE_STATUS_SUCCESS'
22 GROUP BY public_key_algorithm, signature_algorithm,
23          alt_signature_algorithm, alt_public_key, extended_key_usage_set,
24          key_usage_set, subject_country, ct_entry_log, certificate_type,
25          trust_status;
```

Listing 5.3: SQL statement used to evaluate the data set.

Since the values for alternative public keys were not parsed by *Censys*, the database contains only the raw hex values of the alternative subject public keys. Therefore, the valid entries have been clustered according to the length of the raw value, as shown in the listing 5.4. Furthermore, all representative values were collected into an array and attached to the category for later processing.

```

1 INSERT INTO pki_analysis
2 INSERT INTO alt_public_key_analysis
3 SELECT length(arrayFirst(tuple -> tuple.id = '2.5.29.72', parsed.
   unknown_extensions).value) AS alt_public_key_raw_size,
4       upper(arrayFirst(entry -> (length(entry) = 2), parsed.subject.
   country)) AS subject_country,
5       arrayFirst(x -> true, splitByChar('_', arrayFirst(x -> true, ct.
   entries.key))) AS ct_entry_log,
6       certificate_type      AS cert_type,
7       trust_status          AS trusted,
8       COUNT(*)              AS distribution,
9       groupArray(arrayFirst(tuple -> tuple.id = '2.5.29.72', parsed.
   unknown_extensions).value) AS alt_public_key_list,
10      today()                AS query_date
11 FROM certificates FINAL
12 WHERE alt_public_key_raw_size > 0
13 GROUP BY subject_country, ct_entry_log, cert_type, trusted,
   alt_public_key_raw_size

```

Listing 5.4: SQL statement used to evaluate alternative public key info.

For the analysis of certificates that cannot be parsed, similar values will be leveraged as in the main analysis, as the listing 5.5 shows. The evaluation of the CT logs is identical. In addition, there is the timely evaluation that combines the year and quarter of the insertion. Lastly, since there are two possible values for failed parsing, they will also be distinguished.

```

1 INSERT INTO pki_analysis_corrupt
2 SELECT arrayFirst(x -> true, splitByChar('_', arrayFirst(x -> true, ct.
   entries.key))) AS ct_entry_log,
3       toYear(inserted_at) AS insert_year,
4       toQuarter(inserted_at) AS insert_quarter,
5       parse_status AS parse_status,
6       COUNT(*)      AS distribution,
7       today()        AS query_date
8 FROM certificates FINAL
9 WHERE parse_status <> 'CERTIFICATE_PARSE_STATUS_SUCCESS'
10 GROUP BY ct_entry_log, insert_year, insert_quarter, parse_status;

```

Listing 5.5: SQL statement used to evaluate certificates that cannot be parsed.

5.5 Visualization

After counting the various combinations, the results need to be visualized to get usable insights. For this purpose, the data is being delivered to a web application throughout a backend REST API.

5.5.1 Backend-Proxy

The main task of the backend application is to enhance and deliver the analysis results from the database to the frontend application.

For this thesis, the enhancement consists of two functionalities. First, the main database stores the numeric OID of the algorithms used. The curated lists of OIDs (cf. Tables C.1 C.2 C.3 C.4 C.5 C.6) are stored and managed in a *MongoDB* instance. The backend uses this mapping to retrieve the common names of the algorithms used.

For alternative public keys, an additional step needs to be performed before retrieving the algorithm name. The algorithm OID and the key size have to be retrieved from the raw hex values of the key. In doing so, the byte values are interpreted according to the ASN.1 structure defined in Listing 5.6. Since the sequence first lists the algorithm OID, the numerical values are parsed into a textual representation of the OID. The resulting numerical OID can then be used to retrieve the common name. If the algorithm used is based on RSA, the BIT STRING of the public key is extracted to retrieve the size of the key. If the algorithm used is based on ECC, the parameters provided by the algorithm identifier are parsed to retrieve the OID of the curve used.

```

1 SubjectAltPublicKeyInfo ::= SEQUENCE {
2     algorithm      AlgorithmIdentifier{{SupportedAlgorithms}},
3     subjectAltPublicKey BIT STRING}
4
5 AlgorithmIdentifier{ALGORITHM:SupportedAlgorithms} ::= SEQUENCE {
6     algorithm      ALGORITHM.&id({SupportedAlgorithms}),
7     parameters     ALGORITHM.&Type({SupportedAlgorithms}{@algorithm})
8     OPTIONAL,
9     ... }

```

Listing 5.6: ASN.1 definition of Subject Public Key Info in ITU X.509 [27] that is also reused for alternative subject public keys

5.5.2 Frontend Implementation

The frontend implementation is an Angular based web application. The content is divided into two main areas. First, there is the aggregation of the six core attributes as seen in Figure 5.2. Those charts can be adjusted using the various filter options above them. Those filters mostly consist of the meta attributes described in section 4.1.3. In addition, convenience filters, such as aggregating low occurrence values or only showing PQC algorithms, were implemented.

Figure 5.3 shows the second area of the web application. There are static evaluations of the different certificates types and their distributions with respect to the other metadata filters. In addition to the static charts, the last collapsible area contains the charts for the evaluation of corrupt certificates.

5.6 Trouble Shooting and Lessons Learned

During the implementation and first execution trials, various obstacles occurred. Although some problems were mitigated, others could not be solved within the thesis deadline.

5 Implementation

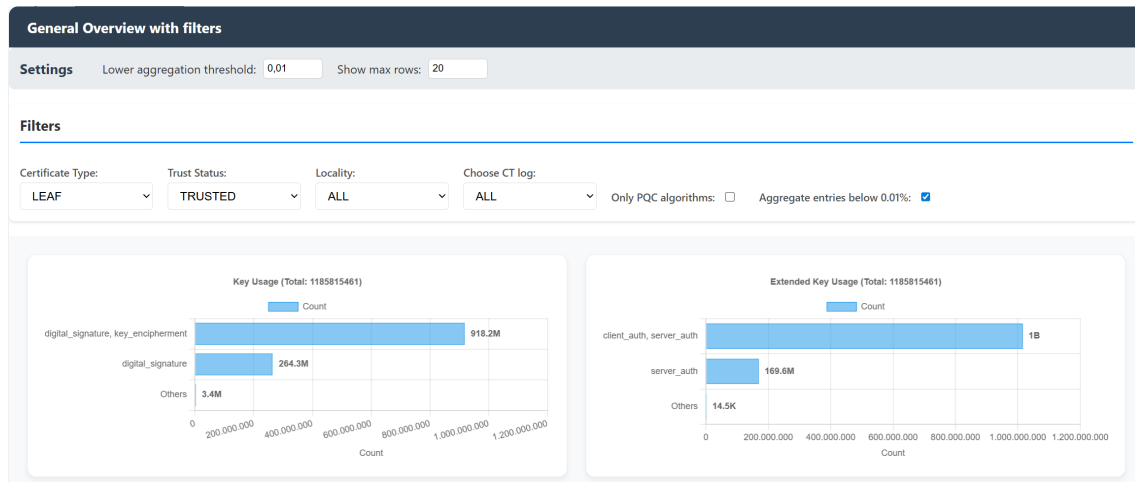


Figure 5.2: Excerpt of PKI Explorer showing the first two general distribution charts, that can be manipulated by setting various filter options

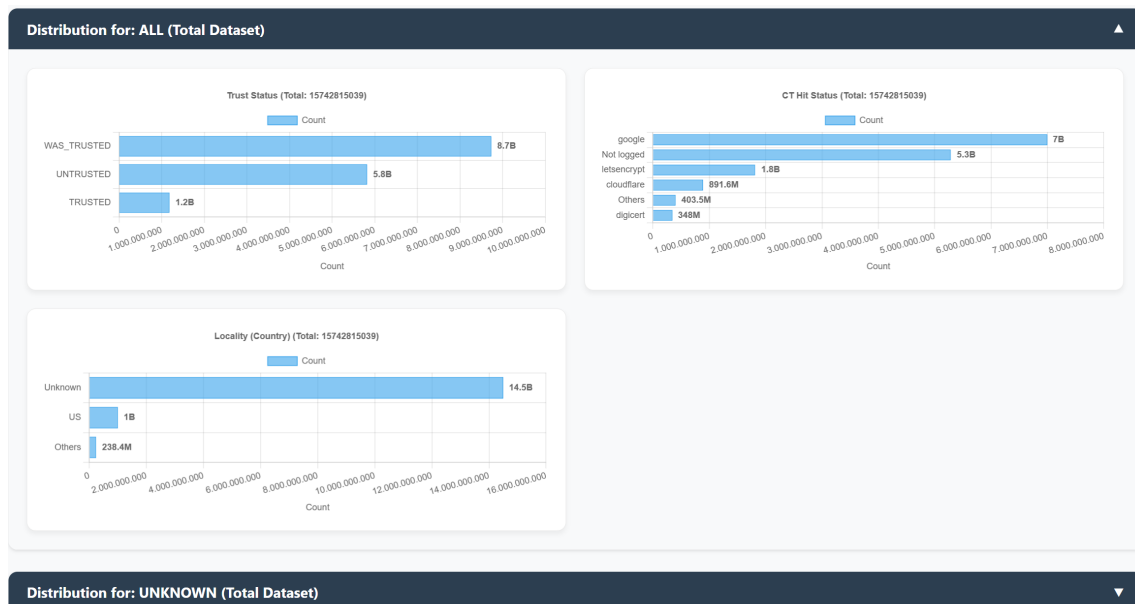


Figure 5.3: Excerpt of PKI Explorer showing the first two collapsible chart groups for each certificate type containing the distribution of the metadata filters

The processing and storing of the *Censys* data on the hardware caused the most problems. This primarily led to cuts in the realized implementations, since damage control took a lot of time.

5.6.1 Hardware Issues

Although *Censys* gave an estimate about the size of the raw data set, the available disk space of the main data server ran out quickly. *Censys* quotes their data size at around 15 TB after cutting it in half using AVRO files for compression. The first time running out of disk space occurred after processing around 12 billion certificates. In response to this, raw certificate values have been excluded from the data processing and consequently were removed from already processed certificates. Since this column was responsible for about half the disk usage with around 14TiB, it occupied nearly half the disk space without providing any additional information. However, after achieving a disk usage of more than 99% again, the data processing had to be stopped. Despite an average compression rate of 2.65:1 TiB by the *Clickhouse* database, the disk space was just not large enough. This resulted in the premature stop of data processing. However, the number of certificates already processed seemed large enough for a representative data basis.

Furthermore, after starting with the evaluation, the SQL statements failed. The database was not able to decompress the data from certain files, throwing a low level I/O error. Presumably a single hard drive was affected by bad sectors. At this point in time, there were no backups, as it was directly after stopping data processing. The column-oriented database structure reduced the affected areas to single columns, such as `fingerprint_sha256` or entry keys of the CT logs. Therefore, the primarily planned analysis was still executed on the entire data set. However, updating single values was for now out of scope, as the primary key column was partially affected. Although the server is secured using HW Raid, a backup was first needed in case the hard drive swap leads to even greater data loss due to further complications. The backup upload to the Supermicro server required 10 days due to the large size of 33 TB. However, the `rsync` tools provided an overview of which files were not transferred due to I/O errors. Therefore, it was now possible to detach the affected parts¹² from the database table, allowing analysis on a slightly reduced, but fully functional, data set. The exclusion contains around 100 million rows.

5.6.2 Software Issues

In addition to failing hardware, the software used was also not always ideal. Optimizing the settings of the *Clickhouse* database instance was achieved by iterative configuration adjustments. This led mainly to performance improvements by using the available hardware ideally. Although column-oriented databases do not excel when it comes to insertion speed, they can mostly mitigate this problem by insert batches. Since the data was already batched by *Censys*, there was no need to further batching in the first place. However, the number of files handled by *Clickhouse* increased faster than the database

¹²*Clickhouse* part management manual: <https://clickhouse.com/docs/sql-reference/statements/alter/partition#detach-partitionpart>

could process them. After reaching more than 20.000 parts, a *Clickhouse* safety mechanism throttled inserting instances by rejecting the insert request with an error description `TooManyParts`¹³. This resulted in using asynchronous inserts, using the memory of the server efficiently as possible, and leading to fewer file creations.

Another problem in the setup was the session handling with *Clickhouse*. A single worker instance connects to the database with a single session. However, when the insert statement fails, it can lead to a locked session. This occurred mainly when inserts failed due to timeouts, caused by the overloaded database. Despite using timeouts between insert retries, it did happen that the entire batch was rejected and put back into queue. However, as mentioned above, accessing a download link often will lead to download blocks, causing further delays.

In summary, the overall processing time was stretched with every new problem. Although some problems were reduced through asynchronous inserts or adjusted timeouts, some problems remained. This resulted in a reduced data set that can be evaluated. The gathered certificates only originate from *Censys*, since the initial concept for firmware images had to be omitted. Additionally, around 100 million entries were omitted due to a failing hard drive. For this reason, the actual data set contains of around 15 billion entries, which are further restricted to certificates that were issued just before June 01, 2025.

¹³Hint for `TooManyParts` error: <https://clickhouse.com/docs/community-wisdom/debugging-insights#too-many-parts-error>

6 Analysis

The processed data sums up to 17.13 billion certificates. During the analysis, it was observed that around 1.39 billion certificates did not carry parsed certificate attributes, resulting in signature algorithms without name or OID. Upon further investigation, it turned out that those certificates carry a different parse status than the default value `CERTIFICATE_PARSE_STATUS_SUCCESS`. Therefore, first the successfully parsed part of the data set will be analyzed according to the proposed attributes. This data set has a remaining size of 15.74 billion certificates. Since the raw certificate values have been dropped, the remaining corrupted certificates will be analyzed to determine where *Censys* found them.

6.1 General Overview

The main certificate data set can be categorized along the classical hierarchy of X.509 certificates. Figure 6.1 shows that 417 certificates have been marked as root CAs. Another 7.3k certificates are marked as intermediate CAs. At the bottom of the hierarchy, there are around 10 billion leaf certificates. Lastly, the remaining 5.8 billion certificates cannot be definitively classified, resulting in an unknown category.

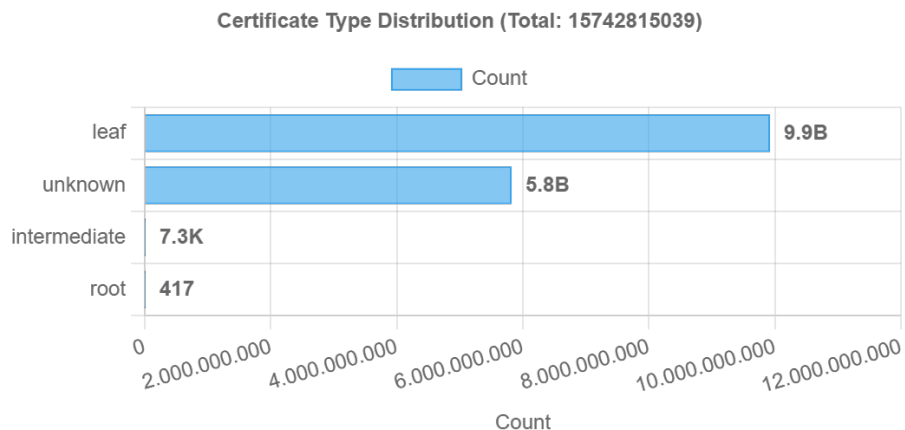


Figure 6.1: *Distribution of certificate types*

6.1.1 Root Certificates

Further evaluation of root CAs reveals that 334 certificates have been marked as trusted. The remaining 83 certificates were once trusted but have already expired, resulting in a trust status was `trusted`. Furthermore, Figure 6.2 shows that 372 certificates have a valid entry in a CT log. Most entries were found in logs from *Google* or *Comodo*.

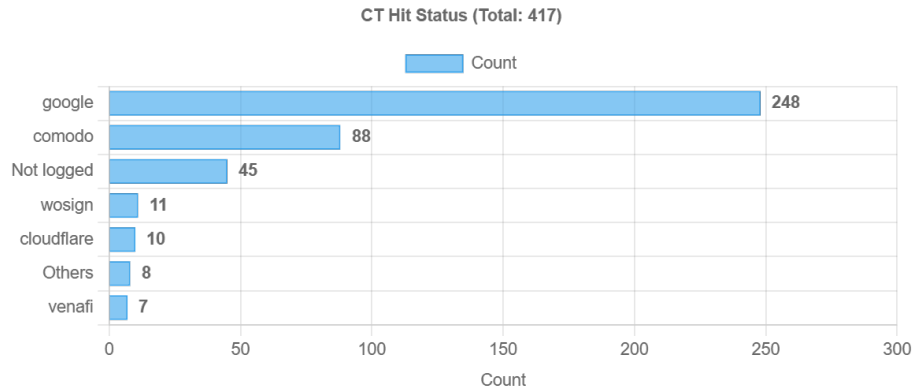


Figure 6.2: CT log distribution of root certificates

Lastly, Figure 6.3 states that approximately one third of root certificates are issued to domains located in the United States. Furthermore, only 16 CAs have been issued, where the country attribute is not in accordance with ISO 31-66-1.

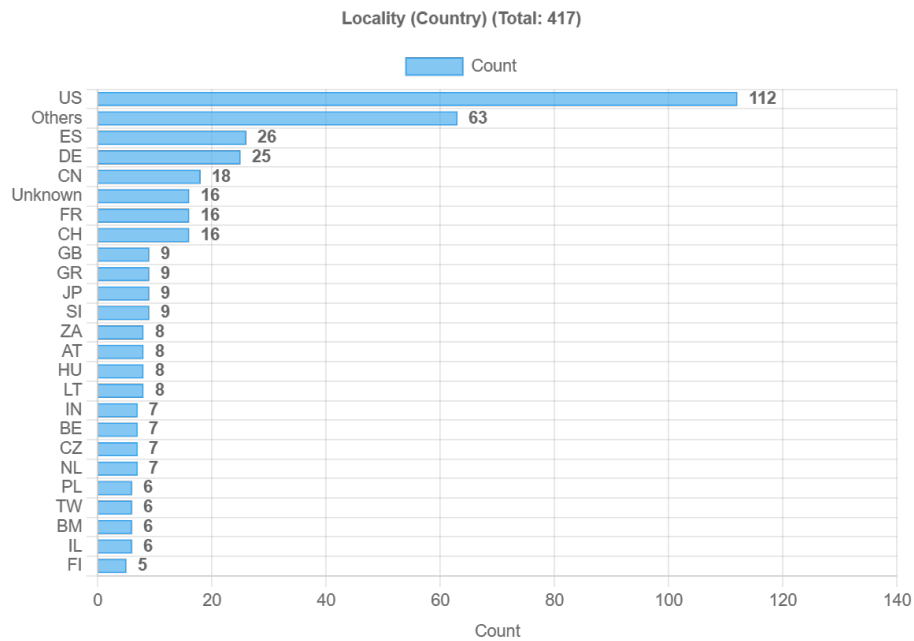


Figure 6.3: Locality distribution of root certificates

6.1.2 Intermediate Certificates

Similar distributions can be found for intermediate certificates. The subset can be divided into 60% trusted and 40% was trusted CAs. Figure 6.4 shows that intermediate certificates pose a high presence in CT logs with a coverage of 98%. Half of them were found in *Comodo* logs.

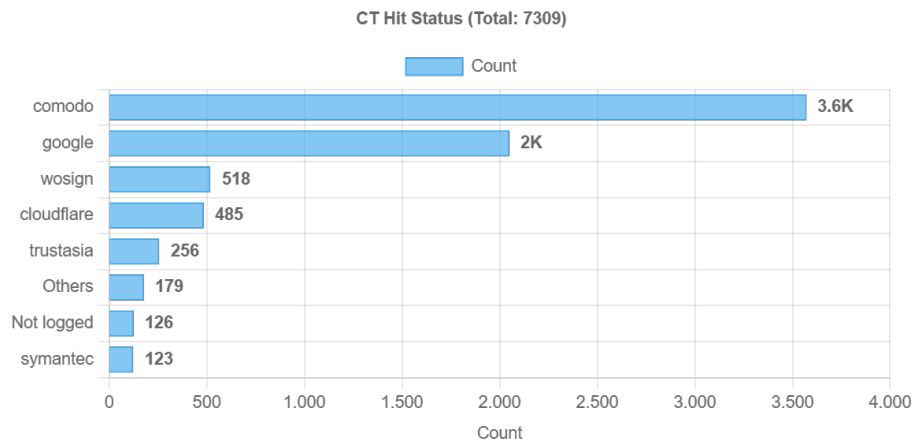


Figure 6.4: CT log distribution of intermediate certificates

Also, similar to root CAs, the United States is responsible for approximately a quarter of intermediate CAs, as shown in Figure 6.5. The percentage of invalid values for the country attribute is low with around 2%.

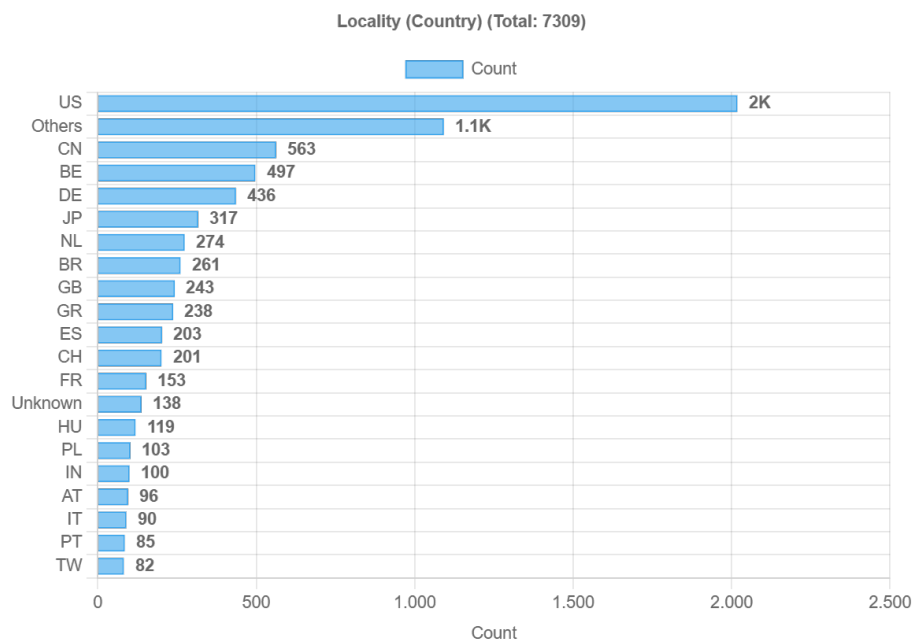


Figure 6.5: Locality distribution of intermediate certificates

6.1.3 Leaf Certificates

At the bottom of the hierarchy, leaf certificates are further evaluated. First, Figure 6.6 shows that there is a much higher turnover rate, as 88% are marked as was trusted.

Of the remaining one billion certificates, only a marginal amount of 19 certificates are considered untrusted, leaving the rest as trusted certificates.

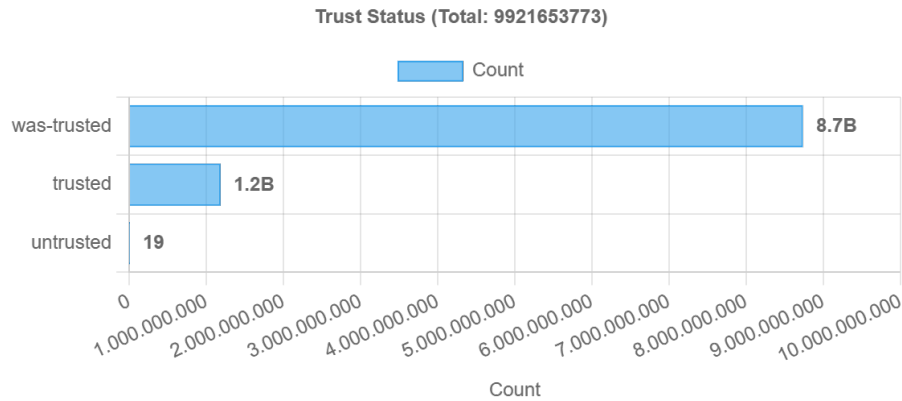


Figure 6.6: Trust status distribution of leaf certificates

Furthermore, the majority of certificates with around 94% are linked to an CT entry. Figure 6.7 shows that most are found in *Google* logs. In combination with logs from *Let's Encrypt* and *Cloudflare* the coverage reaches around 90%.

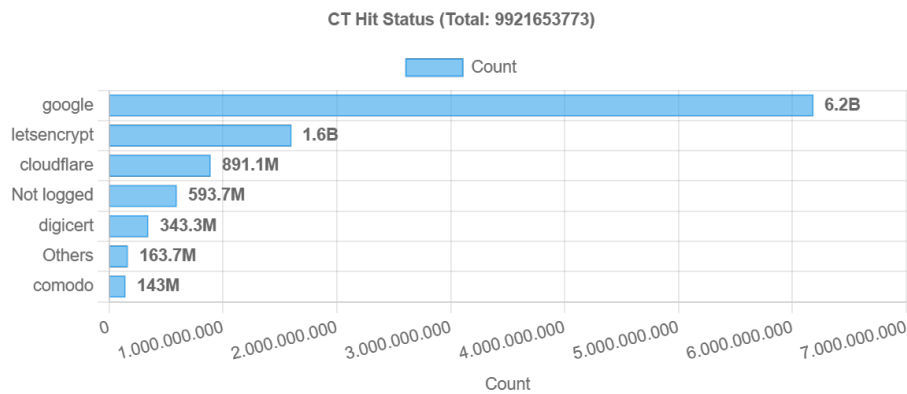


Figure 6.7: CT log distribution of leaf certificates

The evaluation of the locality distribution reveals a contrary trend compared to the CAs. 93% do not have a value for the country attribute according to ISO 31-66-1. Furthermore, nearly 7% are located in the United States.

6.1.4 Unknown Certificates

The evaluation of the trust status for unknown certificates shows that no certificate in this category was ever considered trusted. This reveals most likely the reason why those have not been given a tag by *Censys*. Figure 6.8 shows that only about 20% were found in CT logs, mostly by *Google* or *Let's Encrypt*. Therefore, most of them were found during IPv4 address scans.

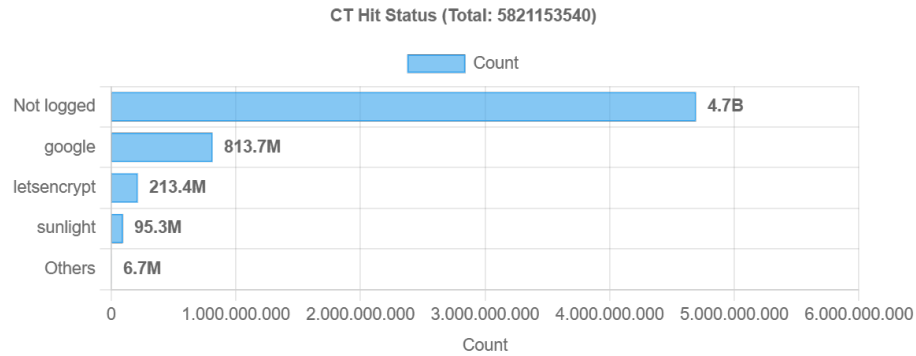


Figure 6.8: CT log distribution of unknown certificates

The locality distribution is also not useful, as 91% contain a value that does not comply with ISO 31-66-1. However, of the remaining certificates, the United States leads the chart, as shown in Figure 6.9.

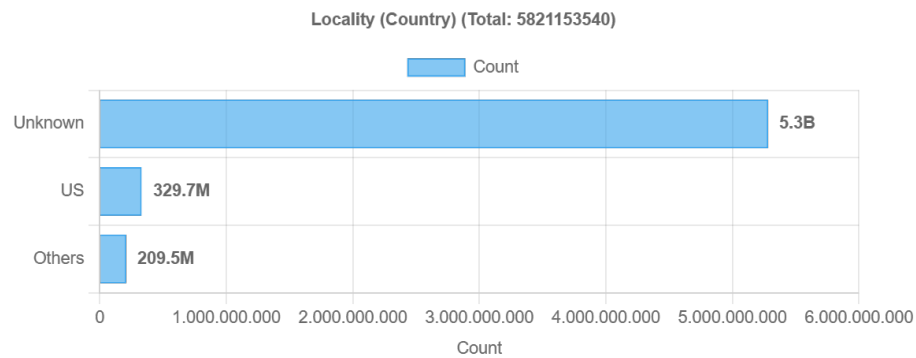


Figure 6.9: Locality distribution of unknown certificates

6.1.5 Summary

The evaluation of the meta information revealed clear trends. For CT logging *Google* and *Comodo* are leading the field for CAs issuance. For leaf certificates *Google* still leads but is now followed by *Let's Encrypt* and *Cloudflare*.

The locality evaluation contains a high percentage of dark figures, especially for leaf certificates. However, the conforming values show a strong focus on certificate issuance in the United States, especially for CAs.

6.2 Key Usage Distribution

Since CAs are often regulated regardless of whether they are a root CA or not, they will be analyzed in combination. However, to avoid distortion by the large amount of

leaf certificates, they will be analyzed separately. Lastly, unknown certificates are also excluded since they most likely contain exceptional values.

6.2.1 Root and Intermediate CAs

Starting with root and intermediate CAs, they normally restrict themselves to a few key usages. The baseline requirements of the CA/B Forum ¹ also state that CA certificates should only carry the values `cRLSign` and `keyCertSign`. Figure 6.10 supports this statement, showing 70% of root certificates that only possess these two attributes. Besides the exception of 21 certificates that carry no key usage attribute at all, the remaining certificates carry the two base attributes with additional attributes, such as `digitalSignature`.

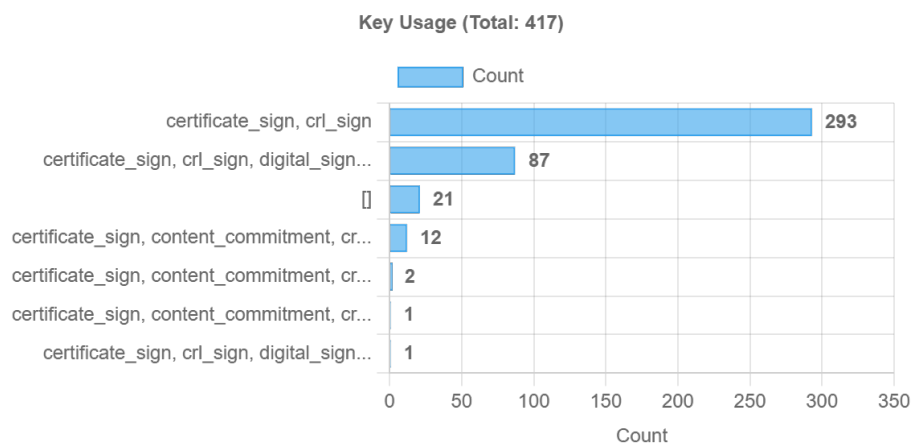


Figure 6.10: Key usage distribution of root certificates

For intermediate CAs Figure 6.11 shows a clear dominance of 98% containing at least `cRLSign` and `keyCertSign` as a key usage attribute. About one half of those certificates additionally carry the value for `digitalSignatures`.

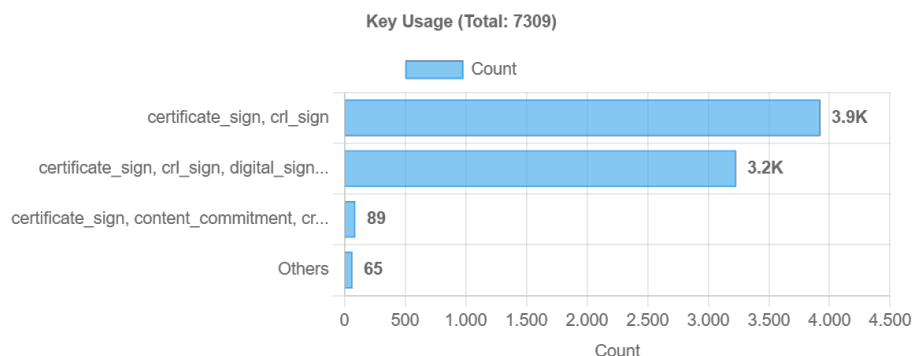


Figure 6.11: Key usage distribution of intermediate certificates

¹CA/B Forum Baseline Requirements: <https://cabforum.org/working-groups/server/baseline-requirements/requirements/>

6.2.2 Leaf Certificates

Moving on to the leaf certificates, there is also a clear leading use case. As Figure 6.12 shows, the combination of `digitalSignature` and `keyEncipherment` is clearly dominating. Most probably since the leaf certificates are the ones used to actually perform encryption and authenticate towards the relying party, this results in the high majority of 81% (or rather 99% adding the certificates with only `digitalSignature`).

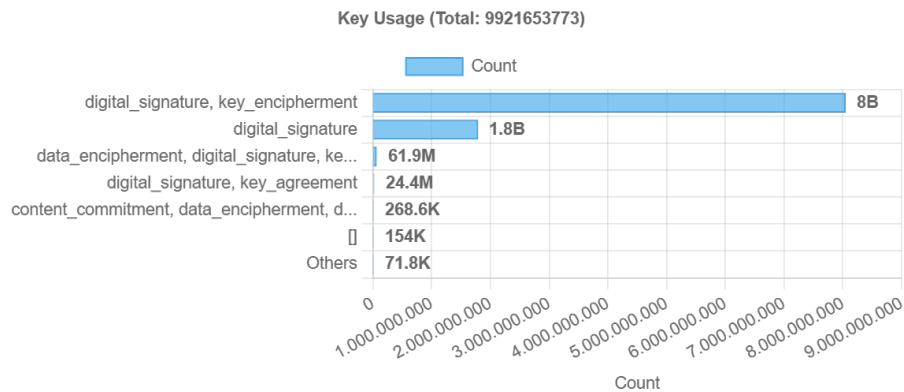


Figure 6.12: Key usage distribution of leaf certificates

6.2.3 Unknown Certificates

Lastly, Figure 6.13 shows a mix of different base types, when taking into account the prior trends. The majority of 51% provide the same combination of key usages, most commonly used by leaf certificates. The second largest subset stands out because no values are provided for key usage. The remaining values may be associated with different levels depending on interpretation.

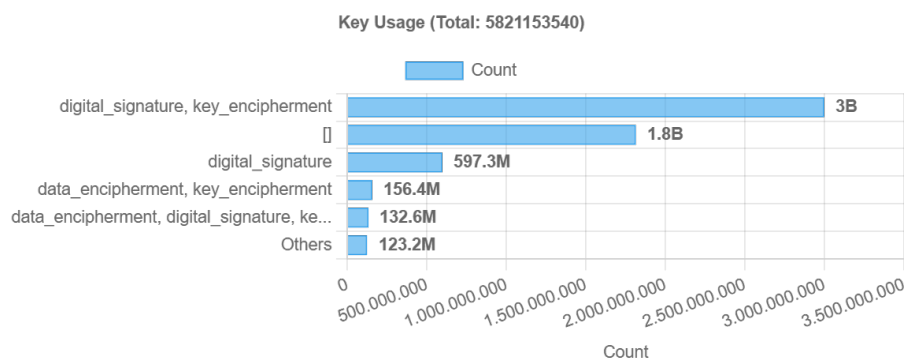


Figure 6.13: Key usage distribution of unknown certificates

6.2.4 Summary

For CAs, a high percentage aligns with the requirements defined by the CA/B Forum. For leaf certificates, most are issued to perform keyEncipherment or are used for digital signatures. Those trends align with the classical values found in the World Wide Web, where the main purpose is found in encryption.

6.3 Extended Key Usage Distribution

Similarly as for the key usage analysis, the extended key usage will be analyzed for CAs, end entities, and unknown certificates separately.

6.3.1 Root and Intermediate CAs

Starting at the top again, nearly no root certificate has values for the extended key usage extension. Figure 6.14 shows that only 6 certificates are attributed with mainly five or more similar values, where only one certificate has not yet expired.

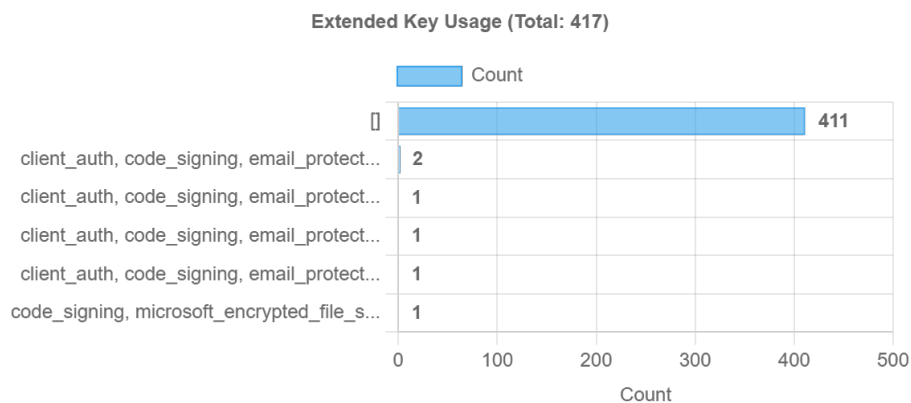


Figure 6.14: *Extended Key usage distribution of root certificates*

Of the remaining CAs, 42% of the intermediate certificates follow this trend, as the Figure 6.15 shows. However, there are also different value combinations such as `clientAuth` and `serverAuth` which takes up 28%. After that, `clientAuth` in combination with `emailProtection` follows with around 8%, representing the mail domain with protocols like *S/MIME*. This indicates that some intermediate CAs are issued for special purposes and not just signing generally.

6.3.2 Leaf Certificates

When evaluating the distribution for leaf certificates shown in Figure 6.16, there is a definite dominance. 89.5% of certificates have the combination of `clientAuth` and `serverAuth`. Additionally, the nearly entire rest has `serverAuth` as attribute value. Therefore, only 0.02% remains, having other values for the extended key usage. Since these

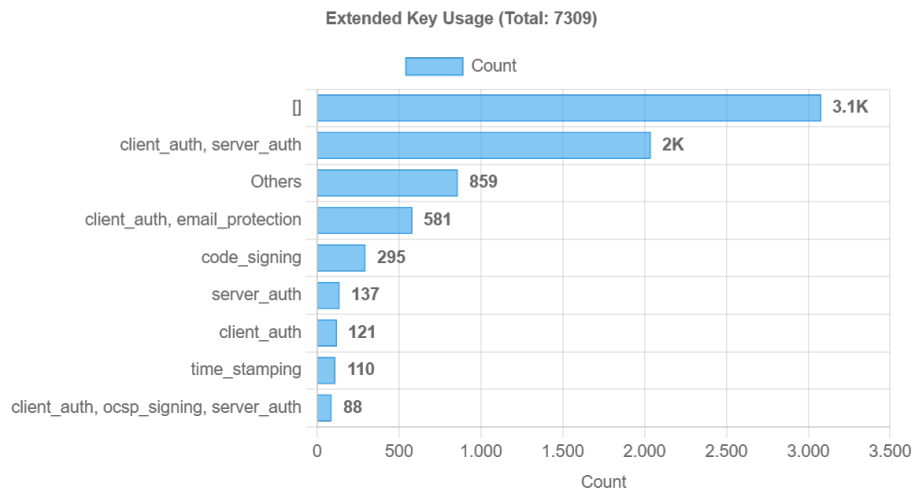


Figure 6.15: Extended key usage distribution of intermediate certificates

combinations are crucial to the *TLS* protocol, this dominance seems plausible. Mostly because *TLS* is one of the most established protocols for data encryption.

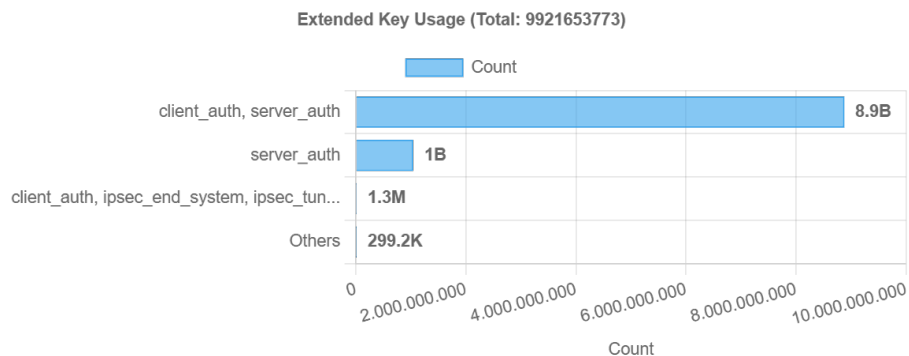


Figure 6.16: Extended key usage distribution of leaf certificates

6.3.3 Unknown Certificates

Lastly, the unknown subset is shown in Figure 6.17. In contrast to the assumptions deducted from the key usages, it displays a vast majority of empty extended key usage extensions. Those values are most commonly known for CAs of higher level. However, the remaining unknown certificates follow the trend of leaf certificates and contain `serverAuth` in combination with `clientAuth`.

6.3.4 Summary

The values for extended key usage are treated differently depending on the CA type. Root CAs do not provide a value, expect some exceptions. However, half of the intermediate CAs

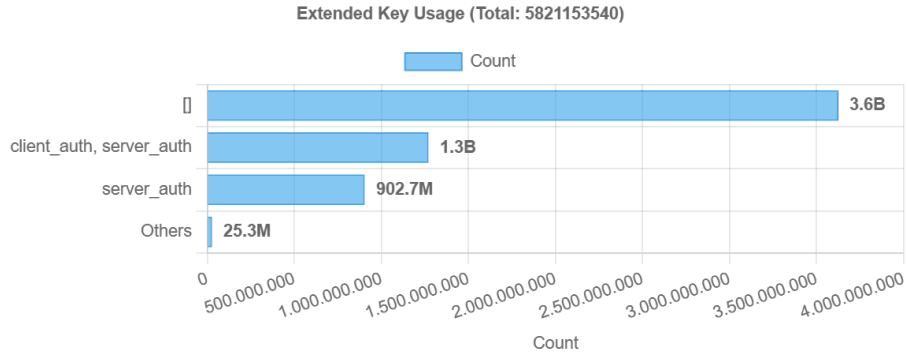


Figure 6.17: *Extended key usage distribution of unknown certificates*

start to provide value combinations. In doing so, they provide more variants in contrast to leaf certificates, which mainly use the combination of `clientAuth` and `serverAuth`.

6.4 Public Key Algorithm Distribution

For algorithm distributions, this thesis first evaluates general trends before looking at the progression of PQC algorithms.

6.4.1 General Algorithm Distribution

The first attribute with cryptographic algorithms are the subject public keys. Figure 6.18 shows that `RSAES-PKCS1-v1_5` dominates in all levels of hierarchy. Especially, RSA keys with a size of 2048 bits are the most popular. The second largest part is using algorithms based on ECC. Furthermore, almost all of them are based on the P-256 curve. The remaining 0.01% mostly belong to the unknown category or are expired certificates using deprecated algorithms such as Digital Signature Algorithm (DSA).

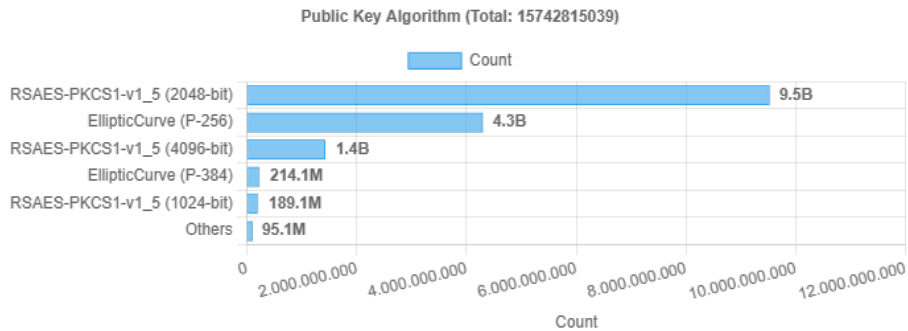


Figure 6.18: *Public key algorithm distribution over all certificate types.*

Furthermore, Figure 6.19 shows that CAs, especially root CAs, tend to use larger keys. The majority of the RSA keys prefer to use a key size of 4096 bits. For algorithms based on ECC, the P-384 curve is favored over the P-256 curve.

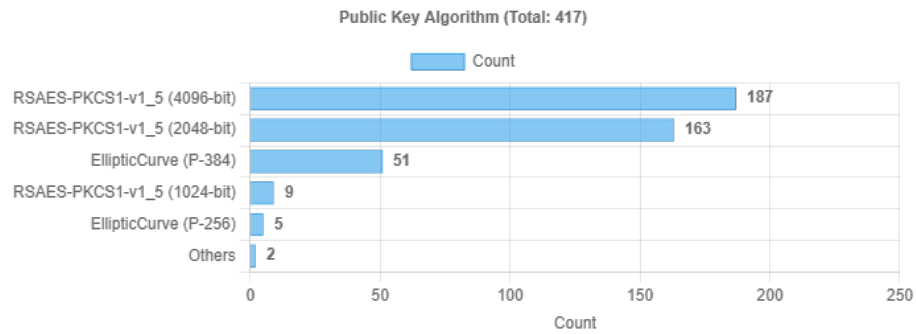


Figure 6.19: Public key algorithm distribution of root certificates

6.4.2 Distribution of PQC Algorithms

In the small subset of outliers, 35 representatives of PQC algorithms can be found. Figure 6.20 shows a mix of standardized and initial draft PQC algorithms. Omitting entries using the former OIDs or standardized algorithms, only 20 entries remain using different levels of ML-DSA. Furthermore, all hits come from the unknown category and are marked as untrusted. This supports the conclusion that the unknown category contains certificates exposed from test environments.

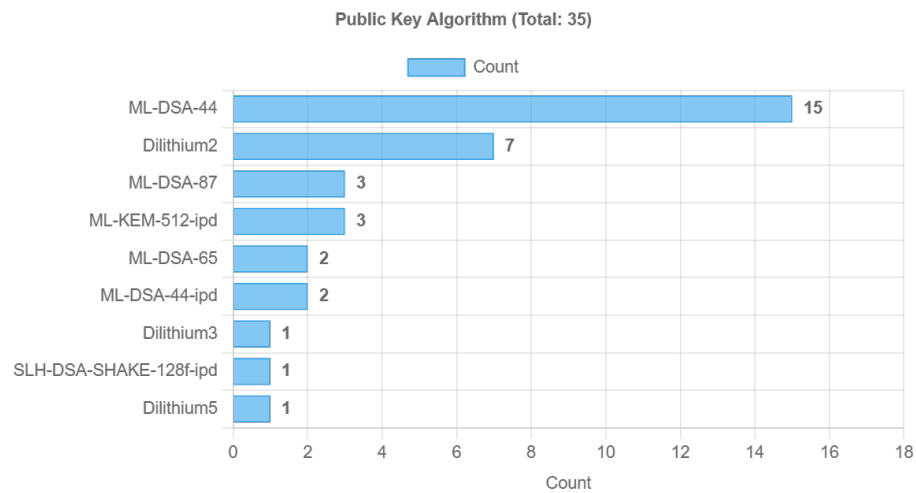


Figure 6.20: Distribution of PQC algorithms as public key algorithm

6.4.3 Summary

The distribution of public keys is dominated mainly by RSAES-PKCS-1 followed by ECC. From the small remaining subset of certificates, some also use PQC algorithms, most of them ML-DSA. However, all of these certificates are marked as untrusted and do not contribute to PQC adoption in productive environments.

6.5 Signature Algorithm Distribution

For signature algorithms, this thesis also distinguishes between general trends and progression in PQC adoption.

6.5.1 General Algorithm Distribution

For signature algorithms, similar trends can be observed in Figure 6.21. Therefore, SHA-256 in combination with RSA is responsible for 83% of all certificate entries. In second and third place are two variants of ECDSA using SHA-384 and SHA-256.

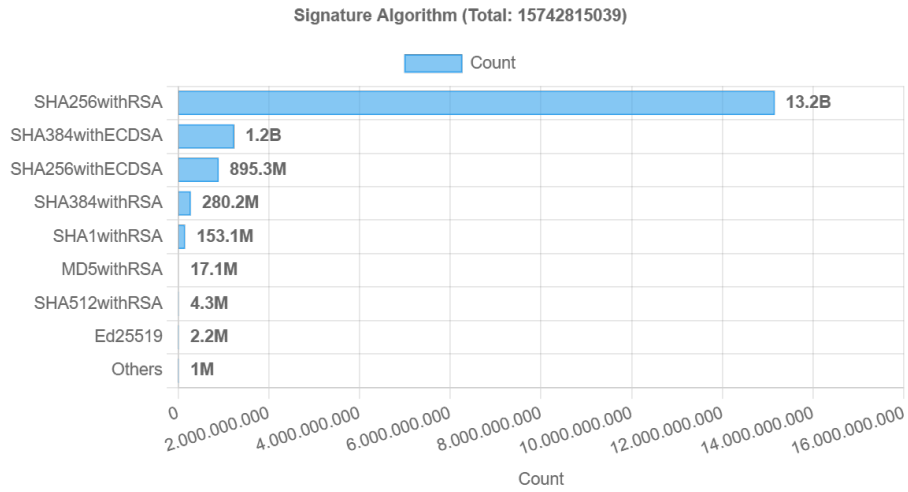


Figure 6.21: Signature algorithm distribution over all certificate types

Furthermore, CAs do not tend to use larger hash functions, in contrast to public keys. In contrast, Figure 6.22 shows that approximately 5% of intermediate CAs are actively signed using SHA-1 with RSA. Although SHA-1 has not been broken yet and deprecation is set to 2030 [37], this indicates a slower transition to safer algorithms.

6.5.2 Distribution of PQC Algorithms

When searching for PQC algorithms, the number is even smaller than for subject public keys, as shown in Figure 6.23. Similarly to the findings for public keys, the only entries belong to untrusted certificates, implying another set of test certificates.

6.5.3 Summary

The signature algorithms show a distribution similar to the public key distribution. High dominance of RSA variants followed by ECC algorithms. However, around 1.4k intermediate certificates were signed using SHA-1, signaling a limping transition to safer algorithms. For adoption towards PQC, no valid certificates were found and even fewer experimental entries were identified.

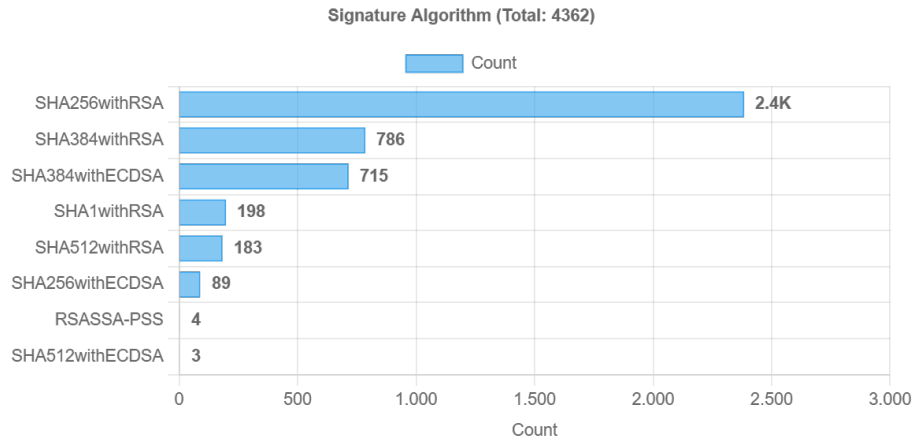


Figure 6.22: Signature algorithm distribution of currently trusted intermediate CAs

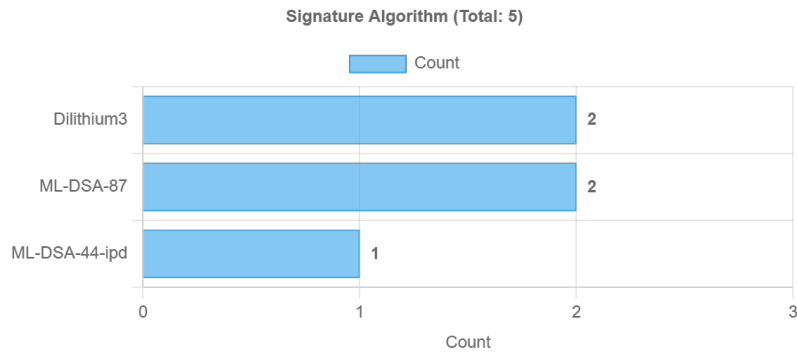


Figure 6.23: Distribution of PQC algorithms as signature algorithm of unknown certificates

6.6 Alternative Signature Algorithm Distribution

The data set gathered seems to contain almost no entry on hybrid certificates. None of the trusted certificates carried a value for the alternative signature extension. Only a small set of 84 unknown certificates with valid entries was found, as shown in Figure 6.24. The fact that only unknown certificates take advantage of hybrid certificates and the concept is overall rather unsupported again supports the claim that unknown certificates contain results from test environments. Nevertheless, since hybrid certificates can be used to transition or rollover to any kind of cryptographic algorithm, there are plenty of conventional algorithms. Therefore, the leading algorithm is SHA256-RSA. However, in second place a post-quantum safe ML-DSA-65-ipd can be found.

6.7 Alternative Public Key Algorithm Distribution

Although hybrid certificates need an alternative public key and an alternative signature value, only 77 entries were found. Identical to the alternative signatures, were the only entries found in the untrusted certificates. Figure 6.25 shows similarity to all other

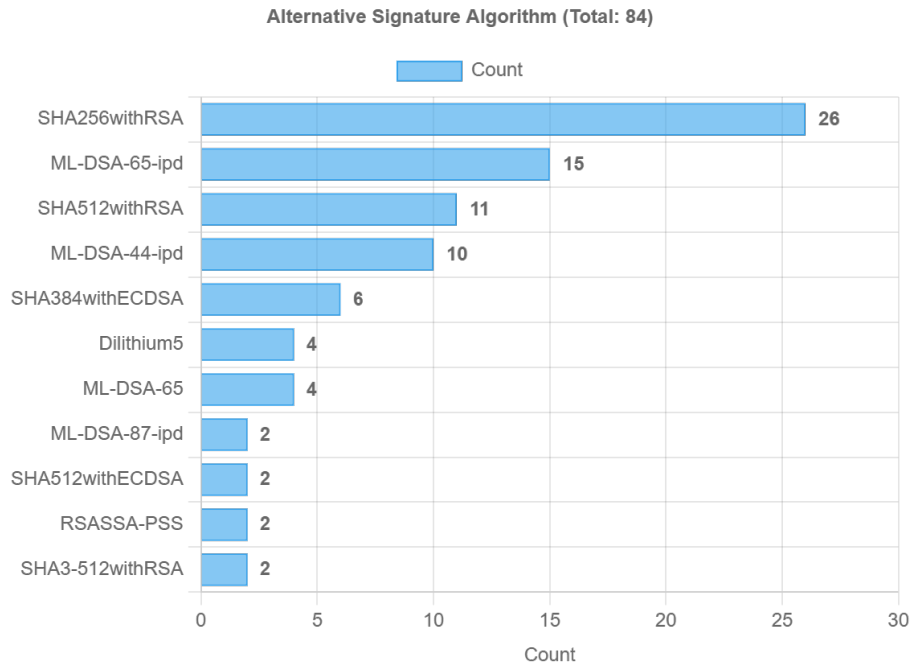


Figure 6.24: Alternative signature algorithm distribution of unknown certificates

algorithm distributions with a majority of RSA variants. Furthermore, it can also be observed that some experimental certificates have been issued using PQC algorithms. However, most of them use OIDs from the initial public draft for ML-DSA.

6.8 Corrupted Certificates

First, it was evaluated which different kinds of parse statuses exist in addition to the default value. Figure 6.26 shows two further categories: `CERTIFICATE_PARSE_STATUS_CORRUPTED` and `CERTIFICATE_PARSE_STATUS_FAIL`. Without further information provided by *Censys*, it seems that most of those certificates can actually not be parsed due to various kinds of corruption and only a small percentage failed due to *Censys*.

Figure 6.27 provides a timeline of when the certificates were first inserted by *Censys*. The first occurrences were found in the second quarter of 2023. Furthermore, until the third quarter of 2024 the numbers were relatively low with a couple of thousands. Since then, the amounts have increased drastically to around several hundred millions per quarter. However, because the initial data set was not completely processed, the final numbers of the second quarter of 2025 are not complete. Therefore, it cannot be derived whether those numbers are increasing even further.

Lastly, the origin of the various corrupt certificates is evaluated. Figure 6.28 shows that the smaller subset of failed certificates was primarily found during IPv4 scans. Only the CT logs maintained by *Google* contain a small part of 1.9k certificates.

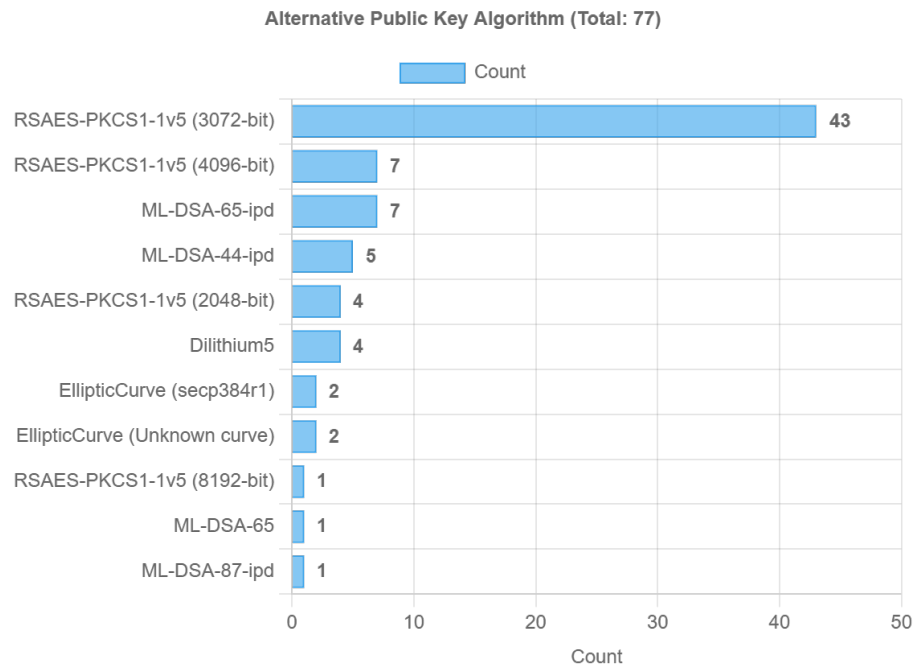


Figure 6.25: Alternative public key algorithm distribution of unknown certificates

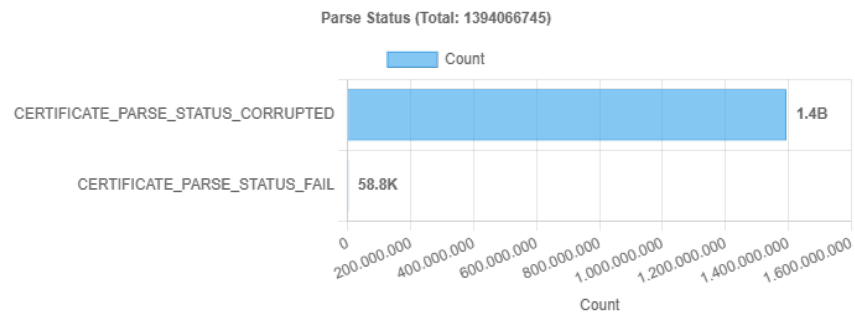


Figure 6.26: Parse status distribution of corrupt certificates

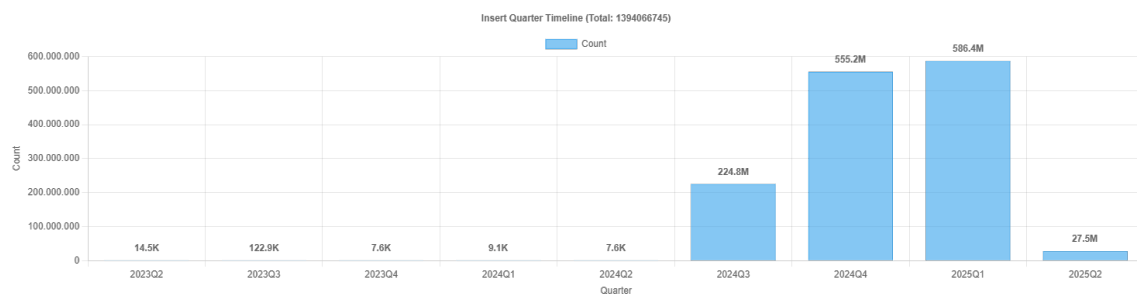


Figure 6.27: Timeline of corrupt certificate insertions

However, the actual corrupted certificates originate mainly from CT logs. Figure 6.29 shows that those occurrences can be reduced to the two log maintainers: *Let's Encrypt* and

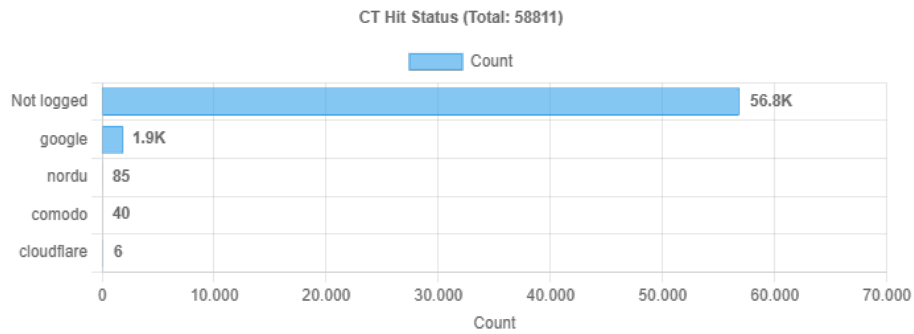


Figure 6.28: CT log distribution of corrupt certificates with status *FAIL*

Sunlight, which is also maintained by *Let's Encrypt*². This information clearly identifies *Let's Encrypt* as the source of the corruption.

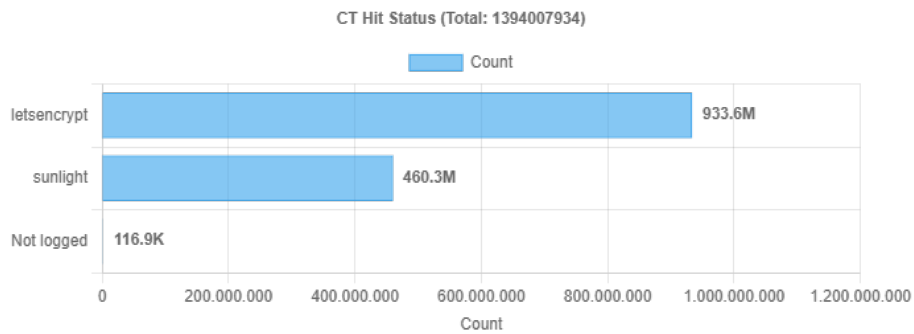


Figure 6.29: CT log distribution of corrupt certificates with status *CORRUPTED*

The analysis of corrupted certificates showed an increasing presence since the third quarter of 2023. Furthermore, most of them were found in CT log instances maintained by *Let's Encrypt*.

²Press release on the relation between Let's Encrypt and Sunlight: <https://letsencrypt.org/2025/06/11/reflections-on-a-year-of-sunlight>

7 Discussion

After evaluating the data set, the results will be discussed in more detail and eventually compared with the results gathered by similar works. This shall either support the findings of this thesis or challenge certain decisions of this thesis or the object of comparison.

7.1 Data Quality

The quality of the evaluated data is strictly correlated with the limitations exhibited. First, the data set only contains certificates from PKIX observable by *Censys*. Therefore, the evaluation results can only show the trends for this sector. This results in leaving out other sectors, such as firmware images, as an essential part of the global PKIX ecosystem. Furthermore, the time-based limitation to certificates before June 01, 2025, in combination with the entries lost to hardware failures, also affects the significance of the generated results. Especially, the analysis of PQC adoption would have benefited from newer data up to January 01, 2026. Nevertheless, the evaluation of 15 billion certificates provided valuable information for science.

The certificate data provided by *Censys* is moderately mixed. Especially, the high number of certificates that cannot be parsed is raising a lot of questions. During the preparation phase, no information was found indicating the existence of such entries. Furthermore, this subset evolved in the middle of 2023, resulting in a relatively short period of time considering the amount. This is also supported by the findings of Stefan Tatschner et al. [57]. They used the same data source provided by *Censys* to perform their analysis on parsing behavior. They covered the data set up to July 01, 2023. Since they did not mention this special subset, it aligns with the fact that these certificates were created during the last two years.

The remaining certificate data was clearly categorized into common levels of PKIX hierarchy. Additionally, malicious certificates that are considered untrusted by *Censys* end up being undetermined. However, *Censys* only classifies 417 certificates as root CAs. Comparing these values with the findings of Anuar Elio Magliari [34], indicate some discrepancies. During his thesis, 5 million certificates were analyzed, resulting in 1974 distinct root CAs. This discrepancy may result from the unknown logic behind the tags issued by *Censys*. Another reason could be that an essential part of the root certificates was lost due to a partially broken hard drive.

7.2 Use Case Trends

The analysis of both key usage attributes rather confirmed the findings of related work than revealed new insights. Since the public PKIX covered by *Censys* contains a high percentage of the Web PKI, TLS is the most prominent usage. Comparison with data from private PKIX implementations would probably create a more diverse use case distribution.

Nevertheless, the evaluation found a relatively high percentage of intermediate CAs providing email_protection as an extended key usage value. This flag is normally associated with the S/MIME protocol. However, there were hardly any related end entity certificates. This may be explained by considering the results found by Gurur Öndarö et al. [43]. They found a much larger data set of 41 million S/MIME certificates. However, they had to search deeper than the traditional IPv4 address space. They evaluated identified LDAP servers to retrieve most of their data set. This concludes that *Censys* currently is able to locate LDAP instances, but does not look further into those data structures, although they may contain even more certificates.

7.3 Algorithm Trends

The evaluation of public key and signature algorithms showed a definite dominance of algorithms based on RSA. Furthermore, an increase in the size of the public key was observed towards higher CAs. When comparing the findings with the work of Anuar Elio Magliari [34], similarities can be found. The general trends are identical, with RSA emerging as the leading cryptographic system. Furthermore, the results of the comparison also found specific trends for different hierarchy levels, such as the larger size of the keys used by root CAs. Considering the fact that this thesis evaluated 500 times more certificates than the comparing work, confirms the found trends of A. E. Magliari.

7.3.1 PQC Adoption

Evaluation of PQC adoption in the PKIX ecosystem showed almost no activity in this field. One year after standardization by NIST, no trusted certificate was found in the data set. Considering the urgency expressed by various institutions such as NIST or the german federal office for information security, these results are rather uprising. Furthermore, the information provided by the PQCCM[45], shows that most vendors for PKIX implementations are already supporting various PQC approaches. However, the matrix does not provide historical evolution. Therefore, it could be possible that the vendors achieved PQC readiness shortly after the time-based limitations of the evaluated data set. Nevertheless, up until June 01, 2025, the industry has not taken the initiative to adopt PQC algorithms.

8 Conclusion & Future Work

This thesis was able to evaluate various trends in X.509 certificates, focusing on PQC adoption and general use case trends. The main contribution of this work is the development of *PKI Explorer*. A landscape of tools capable of maintaining large data sets, efficiently evaluating various attributes, and visualizing the results with potential filter possibilities.

However, the final scope of the data analyzed was greatly reduced due to various complications. Therefore, only public PKIs certificates have been analyzed, using *Censys* as data provider. Evaluation of alternative data sources, such as firmware images, was not possible due to a lack of disk space and timely manners. This also led to the fact that the data set provided by *Censys* was only analyzed up to June 01, 2025.

Nevertheless, the results generated were able to answer most of the research questions, at least from the perspective of the public PKIs. For RQ1, this thesis identified that CAs focuses on certificate issuance as intended by ITU X.509 [27]. Furthermore, intermediate CAs sometimes carry additional information about the use case by leveraging the extended key usage extension. Therefore, after the majority of general signature instances, smaller subsets specialize in `clientAuth` & `serverAuth` and `clientAuth` & `emailProtection`. For leaf certificates, the findings show a clear dominance of `keyEncipherment` and `digitalSignatures`. In combination with the leading values from the extended key usages, those certificates are mostly intended to conventionally secure internet traffic using TLS in client-server applications. This contains authentication by the server instance, and hence the `digitalSignature` purpose. In the second step, data is encrypted using a session key, which is derived from the `keyEncipherment` value. Furthermore, those results lead to even more insight. The definite trends for leaf certificates indicate that the observable public PKIX aligns strongly with the definition of Web PKI. Although different purposes are indicated by intermediate CAs, these cannot be found in leaf certificates.

For RQ2, the current trends of various algorithm identifiers were analyzed in general. The findings show that the RSA system is the most dominant cryptographic system. For public key information, the algorithms used rely on large enough key sizes, whereas 2048-bit is still the majority. In second place for public keys, ECC. For signature algorithms, RSA also takes the lead, mostly in combination with SHA-256. In the second place ECDSA is used for signatures with SHA-384 or SHA-256.

Lastly, for RQ3, it was actively searched for PQC algorithms. However, although institutions such as NIST already recommend transitioning to PQC, no trusted instances were found up to June 01, 2025. This leads to the answer that the PQC adoption rate is 0% in trusted certificates in this data set. Only a few non-classifiable certificates contained PQC algorithms but most probably were fetched from test environments rather than productive systems.

In addition to answering the research questions, a lot of insight was generated about the realized implementation. In particular, the effort needed to maintain a replica of the *Censys* data set was underestimated, leading to cutbacks considering the implementation and range of the findings.

However, optimizing the performance of *PKI Explorer* also poses as the first intention for future work. In addition, there is a large list of potential features to improve the tool even further. Especially the integration of other data sources, especially from private PKIs, may be interesting to derive more use case specific trends and evolutions. Furthermore, although the base structure was made to make the analysis results comparable, the actual feature was never realized. Additionally, the possibilities of this tool landscape are far from being exhausted. The database contains a lot more valuable information about certificates. Future work can use the basis created in this thesis to derive more trends about PKIX implementations. Another option for future work could be to abandon the idea of maintaining a replica of the *Censys* data set. The exhaustive possibilities of cross referencing various metadata filters is mostly only needed partially. Also, it may be possible to cooperate with *Censys* to adjust the data structure for set aggregations on (extended) key usage. Especially since *Censys* recently announced to migrate researcher access to their new platform and to deprecate the legacy search engine¹.

¹*Censys* Researcher Access: <https://docs.censys.com/docs/research-access-to-censys-data>

List of Figures

2.1	Process of symmetric encryption. Adapted from Mohammed Alenezi et al. [3]	4
2.2	Process of asymmetric encryption. Adapted from Mohammed Alenezi et al. [3]	5
2.3	Process of digital signing and verification	5
2.4	Visualization of trust relationships in a PKI. Adapted from ITU-X.509 [27]	8
2.5	Integration of CT into Workflow of Issuing Certificates by [13]	9
2.6	Schematic view of quantum-safe certificate by Congli Wang et al. [60]	12
2.7	Schematic view of hybrid certificate by Congli Wang et al. [60]	12
2.8	Schematic view of composite certificate by Congli Wang et al. [60]	13
5.1	High-level architecture of <i>PKI Explorer</i>	25
5.2	Excerpt of <i>PKI Explorer</i> showing the first two general distribution charts, that can be manipulated by setting various filter options	33
5.3	Excerpt of <i>PKI Explorer</i> showing the first two collapsible chart groups for each certificate type containing the distribution of the metadata filters	33
6.1	Distribution of certificate types	36
6.2	CT log distribution of root certificates	37
6.3	Locality distribution of root certificates	37
6.4	CT log distribution of intermediate certificates	38
6.5	Locality distribution of intermediate certificates	38
6.6	Trust status distribution of leaf certificates	39
6.7	CT log distribution of leaf certificates	39
6.8	CT log distribution of unknown certificates	40
6.9	Locality distribution of unknown certificates	40
6.10	Key usage distribution of root certificates	41
6.11	Key usage distribution of intermediate certificates	41
6.12	Key usage distribution of leaf certificates	42
6.13	Key usage distribution of unknown certificates	42
6.14	Extended Key usage distribution of root certificates	43
6.15	Extended key usage distribution of intermediate certificates	44
6.16	Extended key usage distribution of leaf certificates	44
6.17	Extended key usage distribution of unknown certificates	45
6.18	Public key algorithm distribution over all certificate types.	45
6.19	Public key algorithm distribution of root certificates	46
6.20	Distribution of PQC algorithms as public key algorithm	46
6.21	Signature algorithm distribution over all certificate types	47
6.22	Signature algorithm distribution of currently trusted intermediate CAs	48
6.23	Distribution of PQC algorithms as signature algorithm of unknown certificates	48
6.24	Alternative signature algorithm distribution of unknown certificates	49

List of Figures

6.25 Alternative public key algorithm distribution of unknown certificates . . .	50
6.26 Parse status distribution of corrupt certificates	50
6.27 Timeline of corrupt certificate insertions	50
6.28 CT log distribution of corrupt certificates with status FAIL	51
6.29 CT log distribution of corrupt certificates with status CORRUPTED	51

List of Listings

- 5.1 SQL statement used to create a table for persisting the analysis results. 26
- 5.2 SQL statement used to create a table for persisting the analysis results for
certificates that cannot be parsed. 27
- 5.3 SQL statement used to evaluate the data set. 30
- 5.4 SQL statement used to evaluate alternative public key info. 31
- 5.5 SQL statement used to evaluate certificates that cannot be parsed. 31
- 5.6 ASN.1 definition of Subject Public Key Info in ITU X.509 [27] that is also
reused for alternative subject public keys 32

- B.1 ASN.1 definition of X.509v3 certificates [27] 70

- D.1 Compact SQL statement used to create a table for persisting the certificates. 75

List of Tables

2.1	Key Attributes of a X.509v3 certificate and purpose description, derived from ITU X509 [27]	6
2.2	Different types of CA certificates defined in ITU X.509 Clause 7.4 [27] . . .	7
2.3	Additional categories in the field of PQC with exemplary algorithms that have been considered for standardization by NIST at some point [1]. Algorithms marked with * will be standardized in the near future [2].	11
4.1	Reduced overview of PQC algorithms and the corresponding OID ranges. Appendix C contains lists for complete overview.	20
5.1	Available hardware units, provided by the University of Applied Sciences Munich	24
A.1	Standard Attributes of a Distinguished Name defined in RFC 5280 [11] . .	68
A.2	Basic Extensions defined in RFC 5280 [11]	69
C.1	Curated list of cryptography algorithm OIDs defined in PKCS[36]	71
C.2	Curated list of cryptography algorithm OIDs defined by ANSI [48]	72
C.3	Curated list of cryptography algorithm OIDs used by Russia [10, 54]	72
C.4	Curated list of cryptography algorithm OIDs EdDSA [28]	72
C.5	Curated list of cryptography algorithm OIDs defined NIST[50]	73
C.6	Curated list of cryptography algorithm OIDs defined for IETF Hackathons [26]	74

Bibliography

- [1] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, et al. *Status report on the third round of the NIST Post-Quantum Cryptography Standardization process*. National Institute of Standards and Technology, Sept. 2022. doi: 10.6028/nist.ir.8413-upd1.
- [2] G. Alagic, M. Bros, P. Ciadoux, D. Cooper, Q. Dang, et al. *Status report on the fourth round of the NIST post-quantum cryptography standardization process*. National Institute of Standards and Technology, Mar. 2025. doi: 10.6028/nist.ir.8545.
- [3] M. N. Alenezi, H. K. Alabdulrazzaq, and N. Q. Mohammad. “Symmetric Encryption Algorithms: Review and Evaluation Study”. In: *International Journal of Communication Networks and Information Security (IJCNIS)* 2 (Apr. 2022). doi: 10.17762/ijcnis.v12i2.4698.
- [4] Apache Doris. *Alternative to ClickHouse*. 2026. URL: <https://doris.apache.org/docs/3.x/gettingStarted/alternatives/alternative-to-clickhouse> (visited on 02/06/2026).
- [5] E. Barker. *Recommendation for key management: Part 1 - General*. National Institute of Standards and Technology, May 2020. doi: 10.6028/nist.sp.800-57pt1r5.
- [6] E. Barker. *Transitioning the Use of Cryptographic Algorithms and Key Lengths*. National Institute of Standards and Technology, 2024. doi: 10.6028/nist.sp.800-131ar3.ipd.
- [7] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis. *Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography*. National Institute of Standards and Technology, Apr. 2018. doi: 10.6028/nist.sp.800-56ar3.
- [8] E. Barker, L. Chen, A. Roginsky, A. Vassilev, R. Davis, et al. *Recommendation for pair-wise key establishment using integer factorization cryptography*. National Institute of Standards and Technology, Mar. 2019. doi: 10.6028/nist.sp.800-56br2.
- [9] E. Barker and A. Roginsky. *Transitioning the use of cryptographic algorithms and key lengths*. National Institute of Standards and Technology, Mar. 2019. doi: 10.6028/nist.sp.800-131ar2.
- [10] D. Baryshkov, V. Nikolaev, and A. V. Chelpanov. *Using GOST R 34.10-2012 and GOST R 34.11-2012 Algorithms with the Internet X.509 Public Key Infrastructure*. RFC 9215. Mar. 2022. doi: 10.17487/RFC9215. URL: <https://www.rfc-editor.org/info/rfc9215>.
- [11] S. Boeyen, S. Santesson, T. Polk, R. Housley, S. Farrell, et al. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. RFC 5280. May 2008. doi: 10.17487/RFC5280. URL: <https://www.rfc-editor.org/info/rfc5280>.

- [12] C. Bonnell, J. Gray, D. Hook, T. Okubo, and M. Ounsworth. *A Mechanism for Encoding Differences in Paired Certificates*. Internet-Draft draft-bonnell-lamps-chameleon-certs-07. Internet Engineering Task Force, Oct. 2025. 52 pp. URL: <https://datatracker.ietf.org/doc/draft-bonnell-lamps-chameleon-certs/07/>.
- [13] Certificate Transparency Google Group. *How CT fits into the wider Web PKI ecosystem*. URL: <https://certificate.transparency.dev/howctworks/> (visited on 12/02/2025).
- [14] Clickhouse. *Columnar databases explained*. Sept. 2025. URL: <https://clickhouse.com/resources/engineering/what-is-columnar-database> (visited on 01/23/2026).
- [15] M. Conti, N. Dragoni, and V. Lesyk. "A Survey of Man In The Middle Attacks". In: *IEEE Communications Surveys & Tutorials* 3 (2016). doi: 10.1109/COMST.2016.2548426.
- [16] D. A. Cooper, D. C. Apon, Q. H. Dang, M. S. Davidson, M. J. Dworkin, et al. *Recommendation for Stateful Hash-Based Signature Schemes*. National Institute of Standards and Technology, Oct. 2020. doi: 10.6028/nist.sp.800-208.
- [17] C. Crane. *PKI Architecture: Fundamentals of Designing a Private PKI System*. 2021. URL: <https://www.thesslstore.com/blog/pki-architecture-fundamentals-of-designing-a-private-pki-system/> (visited on 11/24/2025).
- [18] Z. Durumeric, D. Adrian, A. Mirian, M. Bailey, and J. A. Halderman. "A Search Engine Backed by Internet-Wide Scanning". In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. CCS '15. Denver, Colorado, USA: Association for Computing Machinery, 2015. ISBN: 9781450338325. doi: 10.1145/2810103.2813703. URL: <https://doi.org/10.1145/2810103.2813703>.
- [19] C. Ellison. *SPKI Requirements*. RFC 2692. Sept. 1999. doi: 10.17487/RFC2692. URL: <https://www.rfc-editor.org/info/rfc2692>.
- [20] S. M. Farhan and T. Chung. "Exploring the Evolution of TLS Certificates". In: *Passive and Active Measurement*. Ed. by A. Brunstrom, M. Flores, and M. Fiore. Cham: Springer Nature Switzerland, 2023. ISBN: 978-3-031-28486-1.
- [21] H. Hadan, N. Serrano, and L. J. Camp. "A holistic analysis of web-based public key infrastructure failures: comparing experts' perceptions and real-world incidents". In: *Journal of Cybersecurity* 1 (Dec. 2021). ISSN: 2057-2085. doi: 10.1093/cybsec/tyab025. eprint: <https://academic.oup.com/cybersecurity/article-pdf/7/1/tyab025/50476414/tyab025.pdf>.
- [22] M. El-Hajj and P. Beune. "Lightweight public key infrastructure for the Internet of Things: A systematic literature review". In: *Journal of Industrial Information Integration* (2024). ISSN: 2452-414X. doi: <https://doi.org/10.1016/j.jii.2024.100670>.
- [23] G. Hernandez, M. Muench, D. Maier, A. Milburn, S. Park, et al. *FirmWire - Large Scale Analysis Dataset*. Zenodo, May 2022. doi: 10.5281/zenodo.6516030. URL: <https://doi.org/10.5281/zenodo.6516030>.

- [24] J. Höglund. “Public Key Infrastructure and its applications for resource-constrained IoT”. PhD thesis. RISE Research Institutes of Sweden, 2023. ISBN: 978-91-513-1696-3.
- [25] R.-G. Holz. “Empirical analysis of Public Key Infrastructures and investigation of improvements”. English. PhD Thesis - Research external, graduation external. Technische Universität München, 2014.
- [26] IETF-Hackathon. *Object Identifiers for PQC and Composite*. URL: https://github.com/IETF-Hackathon/pqc-certificates/blob/6c5a40b4a23336214f4824c8ddcd737d71ca35ca/docs/oid_mapping.md (visited on 02/06/2026).
- [27] International Telecommunication Union. *Recommendation X.509 (10/19)*. 2019. URL: <https://www.itu.int/rec/T-REC-X.509-201910-I> (visited on 11/24/2025).
- [28] S. Josefsson and J. Schaad. *Algorithm Identifiers for Ed25519, Ed448, X25519, and X448 for Use in the Internet X.509 Public Key Infrastructure*. RFC 8410. Aug. 2018. DOI: 10.17487/RFC8410. URL: <https://www.rfc-editor.org/info/rfc8410>.
- [29] J. Katz and Y. Lindell. *Introduction to Modern Cryptography*. Chapman and Hall/CRC, Aug. 2007. ISBN: 9781420010756. DOI: 10.1201/9781420010756.
- [30] S. Khan, F. Luo, Z. Zhang, F. Ullah, F. Amin, et al. “A Survey on X.509 Public-Key Infrastructure, Certificate Revocation, and Their Modern Implementation on Blockchain and Ledger Technologies”. In: *IEEE Communications Surveys & Tutorials* 4 (2023). DOI: 10.1109/COMST.2023.3323640.
- [31] D. Kim, H. Cho, Y. Kwon, A. Doupé, S. Son, et al. “Security Analysis on Practices of Certificate Authorities in the HTTPS Phishing Ecosystem”. In: *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*. ASIA CCS ’21. Virtual Event, Hong Kong: Association for Computing Machinery, 2021. ISBN: 9781450382878. DOI: 10.1145/3433210.3453100. URL: <https://doi.org/10.1145/3433210.3453100>.
- [32] D. Kim, B. J. Kwon, and T. Dumitraş. “Certified Malware: Measuring Breaches of Trust in the Windows Code-Signing PKI”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’17. Dallas, Texas, USA: Association for Computing Machinery, 2017. ISBN: 9781450349468. DOI: 10.1145/3133956.3133958. URL: <https://doi.org/10.1145/3133956.3133958>.
- [33] B. Laurie, A. Langley, and E. Kasper. *Certificate Transparency*. RFC 6962. June 2013. DOI: 10.17487/RFC6962. URL: <https://www.rfc-editor.org/info/rfc6962>.
- [34] A. E. Magliari. “DigitalCertiAnalytics: A tool for collection and analysis of X.509v3 digital certificates”. MA thesis. Politecnico di Torino, 2025. URL: <https://webthesis.biblio.polito.it/35269/> (visited on 02/06/2026).
- [35] N. van der Meulen. “DigiNotar: Dissecting the First Dutch Digital Disaster”. In: *Journal of Strategic Security* 2 (2013). ISSN: 19440464, 19440472. (Visited on 12/23/2025).
- [36] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch. *PKCS #1: RSA Cryptography Specifications Version 2.2*. RFC 8017. Nov. 2016. DOI: 10.17487/RFC8017. URL: <https://www.rfc-editor.org/info/rfc8017>.

- [37] National Institute of Standards and Technology. 2022. URL: <https://www.nist.gov/news-events/news/2022/12/nist-retires-sha-1-cryptographic-algorithm> (visited on 02/04/2026).
- [38] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. National Institute of Standards and Technology, Feb. 2023. DOI: 10.6028/nist.fips.186-5.
- [39] National Institute of Standards and Technology. *Module-lattice-based digital signature standard*. National Institute of Standards and Technology, Aug. 2024. DOI: 10.6028/nist.fips.204.
- [40] National Institute of Standards and Technology. *Module-lattice-based key-encapsulation mechanism standard*. National Institute of Standards and Technology, Aug. 2024. DOI: 10.6028/nist.fips.203.
- [41] National Institute of Standards and Technology. *NIST Releases First 3 Finalized Post-Quantum Encryption Standards*. 2025. URL: <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards> (visited on 10/14/2025).
- [42] National Institute of Standards and Technology. *Stateless hash-based digital signature standard*. National Institute of Standards and Technology, Aug. 2024. DOI: 10.6028/nist.fips.205.
- [43] G. Öndarö, J. Kaspereit, S. Umezulike, C. Saatjohann, F. Ising, et al. “S/MINE: Collecting and Analyzing S/MIME Certificates at Scale”. In: *Proceedings of the 34th USENIX Security Symposium*. The Advanced Computing Systems Association, 2025. ISBN: 9781939133526. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/oendaroe>.
- [44] S. Paul, Y. Kuzovkova, N. Lahr, and R. Niederhagen. “Mixed Certificate Chains for the Transition to Post-Quantum Authentication in TLS 1.3”. In: *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*. ASIA CCS ’22. Nagasaki, Japan: Association for Computing Machinery, 2022. ISBN: 9781450391405. DOI: 10.1145/3488932.3497755. URL: <https://doi.org/10.1145/3488932.3497755>.
- [45] PKI Consortium. *PQC Capabilities Matrix (PQCCM)*. 2025. URL: <https://pkic.org/pqccm> (visited on 10/20/2025).
- [46] PKIX Working Group. URL: <https://datatracker.ietf.org/wg/pkix/about/> (visited on 02/03/2026).
- [47] S. Pletinckx, T.-D. Nguyen, T. Fiebig, C. Kruegel, and G. Vigna. “Certifiably Vulnerable: Using Certificate Transparency Logs for Target Reconnaissance”. In: *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. 2023. DOI: 10.1109/EuroSP57164.2023.00053.
- [48] T. Polk, R. Housley, S. Turner, D. R. L. Brown, and K. Yiu. *Elliptic Curve Cryptography Subject Public Key Information*. RFC 5480. Mar. 2009. DOI: 10.17487/RFC5480. URL: <https://www.rfc-editor.org/info/rfc5480>.

- [49] M. Raavi, P. Chandramouli, S. Wuthier, X. Zhou, and S.-Y. Chang. “Performance Characterization of Post-Quantum Digital Certificates”. In: *2021 International Conference on Computer Communications and Networks (ICCCN)*. 2021. doi: 10.1109/ICCCN52240.2021.9522179.
- [50] A. Regenscheid and N. Waller. *Computer Security Objects Register - Algorithm Registration*. 2025. URL: <https://csrc.nist.gov/projects/computer-security-objects-register/algorithm-registration#> (visited on 12/29/2025).
- [51] A. Regenscheid. *Transition to Post-Quantum Cryptography Standards*. National Institute of Standards and Technology, 2024. doi: 10.6028/nist.ir.8547.ipd.
- [52] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, et al. *X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP*. RFC 6960. June 2013. doi: 10.17487/RFC6960. URL: <https://www.rfc-editor.org/info/rfc6960>.
- [53] Q. Scheitle, O. Gasser, T. Nolte, J. Amann, L. Brent, et al. “The Rise of Certificate Transparency and Its Implications on the Internet Ecosystem”. In: *Proceedings of the Internet Measurement Conference 2018*. IMC ’18. Boston, MA, USA: Association for Computing Machinery, 2018. ISBN: 9781450356190. doi: 10.1145/3278532.3278562. URL: <https://doi.org/10.1145/3278532.3278562>.
- [54] D. Shefanovski and S. Leontiev. *Using the GOST R 34.10-94, GOST R 34.10-2001, and GOST R 34.11-94 Algorithms with the Internet X.509 Public Key Infrastructure Certificate and CRL Profile*. RFC 4491. May 2006. doi: 10.17487/RFC4491. URL: <https://www.rfc-editor.org/info/rfc4491>.
- [55] A. Sun, J. Lin, W. Wang, Z. Liu, B. Li, et al. “Certificate Transparency Revisited: The Public Inspections on Third-party Monitors”. In: *Proceedings 2024 Network and Distributed System Security Symposium*. NDSS 2024. Internet Society, 2024. doi: 10.14722/ndss.2024.24834. URL: <http://dx.doi.org/10.14722/ndss.2024.24834>.
- [56] S. Tatschner, S. N. Peters, and M. P. Heinl. *Software and Testdata for ParsEval: Evaluation of Parsing Behavior using Real-world Out-in-the-wild X.509 Certificates*. Zenodo, 2024. doi: 10.5281/zenodo.17226796. URL: <https://doi.org/10.5281/zenodo.17226796>.
- [57] S. Tatschner, S. N. Peters, M. P. Heinl, T. Specht, and T. Neue. “ParsEval: Evaluation of Parsing Behavior using Real-world Out-in-the-wild X.509 Certificates”. In: *Proceedings of the 19th International Conference on Availability, Reliability and Security*. ARES ’24. Vienna, Austria: Association for Computing Machinery, 2024. ISBN: 9798400717185. doi: 10.1145/3664476.3669935. URL: <https://doi.org/10.1145/3664476.3669935>.
- [58] P. F. Tehrani, E. Osterweil, T. C. Schmidt, and M. Wählisch. *How to Measure TLS, X.509 Certificates, and Web PKI: A Tutorial and Brief Survey*. 2024. doi: 10.48550/ARXIV.2401.18053. URL: <https://arxiv.org/abs/2401.18053>.

- [59] B. VanderSloot, J. Amann, M. Bernhard, Z. Durumeric, M. Bailey, et al. “Towards a Complete View of the Certificate Ecosystem”. In: *Proceedings of the 2016 Internet Measurement Conference*. IMC ’16. Santa Monica, California, USA: Association for Computing Machinery, 2016. ISBN: 9781450345262. DOI: 10.1145/2987443.2987462. URL: <https://doi.org/10.1145/2987443.2987462>.
- [60] C. Wang, W. Xue, and J. Wang. “Integration of Quantum-Safe Algorithms into X.509v3 Certificates”. In: *2023 IEEE 3rd International Conference on Electronic Technology, Communication and Information (ICETCI)*. 2023. DOI: 10.1109/ICETCI57876.2023.10176713.
- [61] H. Y. Wong. “Shor’s Algorithm”. In: *Introduction to Quantum Computing: From a Layperson to a Programmer in 30 Steps*. Cham: Springer International Publishing, 2024. ISBN: 978-3-031-36985-8. DOI: 10.1007/978-3-031-36985-8_29. URL: https://link.springer.com/chapter/10.1007/978-3-031-36985-8_29.
- [62] Y. Wu, J. Wang, Y. Wang, S. Zhai, Z. Li, et al. “Your Firmware Has Arrived: A Study of Firmware Update Vulnerabilities”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024. ISBN: 978-1-939133-44-1. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/wu-yuhao>.
- [63] S. Yamane, J. Wang, H. Suzuki, N. Segawa, and Y. Murayama. *Rethinking OpenPGP PKI and OpenPGP Public Keyserver*. 2003. arXiv: cs/0308015 [cs.CY]. URL: <https://arxiv.org/abs/cs/0308015>.

Glossary

ACRL Attribute Certificate Revocation List.

AES Advanced Encryption Standard.

AVL Attribute Validation List.

CA Certificate Authority.

CARL Certificate Authority Revocation List.

CBOR Concise Binary Object Representation.

CRL Certificate Revocation List.

CRQC Cryptographically Relevant Quantum Computer.

CT Certificate Transparency.

DES Data Encryption Standard.

DH Diffie-Hellman.

DN Distinguished Name.

DSA Digital Signature Algorithm.

ECC Elliptic Curve Cryptography.

ECDSA Elliptic Curve Digital Signature Algorithm.

EdDSA Edwards-Curve Digital Signature Algorithm.

EES Escrowed Encryption Standard.

FN-DSA fast-Fourier transform over NTRU-Lattice-Based Digital Signature Algorithm.

FORS Forest of Random Subsets.

HNDL harvest now, decrypt later.

HQC Hamming Quasi-Cyclic.

HSM Hardware Security Module.

HSS Hierarchical Signature System.

IoT Internet of Things.

ITU International Telecommunications Union.

LMS Leighton-Micali Signature.

MITM man-in-the-middle.

ML-DSA Module-Lattice-Based Digital Signature Standard.

ML-KEM Module-Lattice-Based Key-Encapsulation Mechanism Standard.

MLWE Module Learning With Errors.

MQV Menezes-Qu-Vanstone.

NIST National Institute of Standards and Technology.

OCSP Online Certificate Status Protocol.

OID Object Identifier.

OLAP online analytical processing.

PKCS Public Key Cryptography Standard.

PKI Public Key Infrastructure.

PKI Consortium Public Key Infrastructure Consortium.

PKIX Public-Key Infrastructure using X.509.

PQC Post-Quantum Cryptography.

PQCCM PQC Capabilities Matrix.

RSA Rivest-Shamir-Adleman.

SCT Signed Certificate Timestamp.

SLH-DSA Stateless Hash-Based Digital Signature Standard.

TBSCertificate To Be Signed Certificate.

XMSS eXtended Merkle Signature Scheme.

A RFC 5280 Definitions

Basic Attributes	Optional Attributes
country	locality
organization	title
organizational unit	surname
distinguished name qualifier	given name
state or province name	initials
common name	pseudonym
serial number	generation qualifier

Table A.1: *Standard Attributes of a Distinguished Name defined in RFC 5280 [11]*

Extension Name	Description
Authority Key Identifier	Identifier for correct signature public key, if multiple concurrent key pair exists
Subject Key Identifier	Identifier for the certificate's public key
Key Usage	Purpose (e.g., encipherment, signature, certificate signing) of certificate's key
Certificate Policies	Policies which apply to the certificate and indicate further purposes
Policy Mappings	Mapping between the policies of issuer and subject (CA only)
Subject Alternative Name	Alternative subject name identifier other than DN (e.g., mail or DNS)
Issuer Alternative Name	Alternative name for the issuer other than DN
Subject Directory Attributes	Break down of additional identification attributes
Basic Constraints	Specifies the boolean flag "cA" and integer "pathLenConstraint" (max allowed sub intermediate CAs)
Name Constraints	Define a valid subject name space for subsequent certificates (CA only)
Policy Constraints	Enforcement and Prohibition of certificate policies and policies mappings (CA only)
Extended Key Usage	Additional or replacing purposes for a certificate
CRL Distribution Points	Information on how CRL information can be obtained
Inhibit anyPolicy	Special policy constraint extension
Freshest CRL	Information on how delta CRL information can be obtained

Table A.2: Basic Extensions defined in RFC 5280 [11]

B X.509v3 Definitions

```
1 Certificate ::= SEQUENCE {
2     tbsCertificate      TBSCertificate,
3     signatureAlgorithm  AlgorithmIdentifier,
4     signatureValue      BIT STRING
5 }
6
7 TBSCertificate ::= SEQUENCE {
8     version              [0] EXPLICIT Version DEFAULT v1,
9     serialNumber         CertificateSerialNumber,
10    signature            AlgorithmIdentifier,
11    issuer               Name,
12    validity             Validity,
13    subject              Name,
14    subjectPublicKeyInfo SubjectPublicKeyInfo,
15    issuerUniqueID       [1] IMPLICIT UniqueIdentifier OPTIONAL,
16                        -- If present, version MUST be v2 or v3
17    subjectUniqueID      [2] IMPLICIT UniqueIdentifier OPTIONAL,
18                        -- If present, version MUST be v2 or v3
19    extensions           [3] EXPLICIT Extensions OPTIONAL
20                        -- If present, version MUST be v3
21 }
22
23 Version ::= INTEGER { v1(0), v2(1), v3(2) }
24
25 CertificateSerialNumber ::= INTEGER
26
27 Validity ::= SEQUENCE {
28     notBefore    Time,
29     notAfter     Time
30 }
31
32 Time ::= CHOICE {
33     utcTime        UTCTime,
34     generalTime    GeneralizedTime
35 }
36
37 UniqueIdentifier ::= BIT STRING
38
39 SubjectPublicKeyInfo ::= SEQUENCE {
40     algorithm      AlgorithmIdentifier,
41     subjectPublicKey BIT STRING
42 }
43
44 Extensions ::= SEQUENCE SIZE (1..MAX) OF Extension
45
46 Extension ::= SEQUENCE {
47     extnID      OBJECT IDENTIFIER,
48     critical    BOOLEAN DEFAULT FALSE,
49     extnValue    OCTET STRING
50                -- contains the DER encoding of an ASN.1 value
51                -- corresponding to the extension type identified
52                -- by extnID
53 }
```

Listing B.1: ASN.1 definition of X.509v3 certificates [27]

C OID Mapping

Although Censys provided clear names for common algorithms such as RSA or common ECC curves, some were only provided as OID. Furthermore, extraction of algorithm information from alternative public key extensions only yielded OIDs. Since no API was found to look up the clear name of a corresponding OID, a static mapping was defined. All values for conventional algorithms were selected because they were relevant in the analysis results.

Algorithm Name	OID
RSAES-PKCS1-v1_5	1.2.840.113549.1.1.1
MD2withRSA	1.2.840.113549.1.1.2
MD4withRSA	1.2.840.113549.1.1.3
MD5withRSA	1.2.840.113549.1.1.4
SHA1withRSA	1.2.840.113549.1.1.5
RSASSA-PSS	1.2.840.113549.1.1.10
SHA256withRSA	1.2.840.113549.1.1.11
SHA384withRSA	1.2.840.113549.1.1.12
SHA512withRSA	1.2.840.113549.1.1.13
SHA512-224withRSA	1.2.840.113549.1.1.14
SHA512-256withRSA	1.2.840.113549.1.1.15

Table C.1: *Curated list of cryptography algorithm OIDs defined in PKCS[36]*

Algorithm Name	OID
EllipticCurve	1.2.840.10045.2.1
SHA256withECDSA	1.2.840.10045.4.3.2
SHA384withECDSA	1.2.840.10045.4.3.3
SHA512withECDSA	1.2.840.10045.4.3.4
SHA1withECDSA	1.2.840.10045.4.3.1
ECDSA-SHA1	1.2.840.10045.4.1
DSA	1.2.840.10040.4.1
SHA1withDSA	1.2.840.10040.4.3
sect163k1	1.3.132.0.1
sect163r2	1.3.132.0.15
sect224r1	1.3.132.0.33
sect233k1	1.3.132.0.26
sect233r1	1.3.132.0.27
sect283k1	1.3.132.0.16
sect283r1	1.3.132.0.17
sect384r1	1.3.132.0.34
sect409k1	1.3.132.0.36
sect409r1	1.3.132.0.37
sect521r1	1.3.132.0.35
sect571k1	1.3.132.0.38
sect571r1	1.3.132.0.39

Table C.2: Curated list of cryptography algorithm OIDs defined by ANSI [48]

Algorithm Name	OID
GOST-R34.10-2012 (256-bit)	1.2.643.7.1.1.1.1
GOST-R34.10-2012 (512-bit)	1.2.643.7.1.1.1.2
GOST-R34.10-2012 (256-bit)	1.2.643.7.1.1.3.2
GOST-R34.10-2012 (512-bit)	1.2.643.7.1.1.3.3
GOST-R34.11-94withGOST-R34.10-2001	1.2.643.2.2.3
GOST-R34.11-94withGOST-R34.10-2001	1.2.643.2.2.4
GOST-R34.10-2001	1.2.643.2.2.19

Table C.3: Curated list of cryptography algorithm OIDs used by Russia [10, 54]

Algorithm Name	OID
Ed25519	1.3.101.112
Ed448	1.3.101.113

Table C.4: Curated list of cryptography algorithm OIDs EdDSA [28]

Algorithm Name	OID
SHA224withDSA	2.16.840.1.101.3.4.3.1
SHA256withDSA	2.16.840.1.101.3.4.3.2
SHA384withDSA	2.16.840.1.101.3.4.3.3
SHA512withDSA	2.16.840.1.101.3.4.3.4
SHA3-224withDSA	2.16.840.1.101.3.4.3.9
SHA3-256withDSA	2.16.840.1.101.3.4.3.10
SHA3-384withDSA	2.16.840.1.101.3.4.3.11
SHA3-512withDSA	2.16.840.1.101.3.4.3.12
SHA3-224withRSA	2.16.840.1.101.3.4.3.13
SHA3-256withRSA	2.16.840.1.101.3.4.3.14
SHA3-384withRSA	2.16.840.1.101.3.4.3.15
SHA3-512withRSA	2.16.840.1.101.3.4.3.16
ML-DSA-44	2.16.840.1.101.3.4.3.17
ML-DSA-65	2.16.840.1.101.3.4.3.18
ML-DSA-87	2.16.840.1.101.3.4.3.19
SLH-DSA-SHA2-128s	2.16.840.1.101.3.4.3.20
SLH-DSA-SHA2-128f	2.16.840.1.101.3.4.3.21
SLH-DSA-SHA2-192s	2.16.840.1.101.3.4.3.22
SLH-DSA-SHA2-192f	2.16.840.1.101.3.4.3.23
SLH-DSA-SHA2-256s	2.16.840.1.101.3.4.3.24
SLH-DSA-SHA2-256f	2.16.840.1.101.3.4.3.25
SLH-DSA-SHAKE-128s	2.16.840.1.101.3.4.3.26
SLH-DSA-SHAKE-128f	2.16.840.1.101.3.4.3.27
SLH-DSA-SHAKE-192s	2.16.840.1.101.3.4.3.28
SLH-DSA-SHAKE-192f	2.16.840.1.101.3.4.3.29
SLH-DSA-SHAKE-256s	2.16.840.1.101.3.4.3.30
SLH-DSA-SHAKE-256f	2.16.840.1.101.3.4.3.31
HASH-ML-DSA-44 (Pre-Hash: SHA-512)	2.16.840.1.101.3.4.3.32
HASH-ML-DSA-65 (Pre-Hash: SHA-512)	2.16.840.1.101.3.4.3.33
HASH-ML-DSA-87 (Pre-Hash: SHA-512)	2.16.840.1.101.3.4.3.34
HASH-SLH-DSA-SHA2-128s (Pre-Hash: SHA-256)	2.16.840.1.101.3.4.3.35
HASH-SLH-DSA-SHA2-128f (Pre-Hash: SHA-256)	2.16.840.1.101.3.4.3.36
HASH-SLH-DSA-SHA2-192s (Pre-Hash: SHA-512)	2.16.840.1.101.3.4.3.37
HASH-SLH-DSA-SHA2-192f (Pre-Hash: SHA-512)	2.16.840.1.101.3.4.3.38
HASH-SLH-DSA-SHA2-256s (Pre-Hash: SHA-512)	2.16.840.1.101.3.4.3.39
HASH-SLH-DSA-SHA2-256f (Pre-Hash: SHA-512)	2.16.840.1.101.3.4.3.40
HASH-SLH-DSA-SHAKE-128s (Pre-Hash: SHAKE-128)	2.16.840.1.101.3.4.3.41
HASH-SLH-DSA-SHAKE-128f (Pre-Hash: SHAKE-128)	2.16.840.1.101.3.4.3.42
HASH-SLH-DSA-SHAKE-192s (Pre-Hash: SHAKE-256)	2.16.840.1.101.3.4.3.43
HASH-SLH-DSA-SHAKE-192f (Pre-Hash: SHAKE-256)	2.16.840.1.101.3.4.3.44
HASH-SLH-DSA-SHAKE-256s (Pre-Hash: SHAKE-256)	2.16.840.1.101.3.4.3.45
HASH-SLH-DSA-SHAKE-256f (Pre-Hash: SHAKE-256)	2.16.840.1.101.3.4.3.46
ML-KEM-512	2.16.840.1.101.3.4.4.1
ML-KEM-768	2.16.840.1.101.3.4.4.2
ML-KEM-1024	2.16.840.1.101.3.4.4.3

Table C.5: Curated list of cryptography algorithm OIDs defined NIST[50]

Algorithm Name	OID
ML-DSA-44-ipd	1.3.6.1.4.1.2.267.12.4.4
ML-DSA-65-ipd	1.3.6.1.4.1.2.267.12.6.5
ML-DSA-87-ipd	1.3.6.1.4.1.2.267.12.8.7
SLH-DSA-SHA2-128s-ipd	1.3.9999.6.4.16
SLH-DSA-SHAKE-128s-ipd	1.3.9999.6.7.16
SLH-DSA-SHA2-128f-ipd	1.3.9999.6.4.13
SLH-DSA-SHAKE-128f-ipd	1.3.9999.6.7.13
SLH-DSA-SHA2-192s-ipd	1.3.9999.6.5.12
SLH-DSA-SHAKE-192s-ipd	1.3.9999.6.8.12
SLH-DSA-SHA2-192f-ipd	1.3.9999.6.5.10
SLH-DSA-SHAKE-192f-ipd	1.3.9999.6.8.10
SLH-DSA-SHA2-256s-ipd	1.3.9999.6.6.12
SLH-DSA-SHAKE-256s-ipd	1.3.9999.6.9.12
SLH-DSA-SHA2-256f-ipd	1.3.9999.6.6.10
SLH-DSA-SHAKE-256f-ipd	1.3.9999.6.9.10
ML-KEM-512-ipd	1.3.6.1.4.1.22554.5.6.1
ML-KEM-768-ipd	1.3.6.1.4.1.22554.5.6.2
ML-KEM-1024-ipd	1.3.6.1.4.1.22554.5.6.3
id-MLDSA44-RSA2048-PSS-SHA256	1.3.6.1.5.5.7.6.37
id-MLDSA44-RSA2048-PKCS15-SHA256	1.3.6.1.5.5.7.6.38
id-MLDSA44-Ed25519-SHA512,	1.3.6.1.5.5.7.6
id-MLDSA44-ECDSA-P256-SHA256	1.3.6.1.5.5.7.6.40
id-MLDSA65-RSA3072-PSS-SHA512	1.3.6.1.5.5.7.6.41
id-MLDSA65-RSA3072-PKCS15-SHA512	1.3.6.1.5.5.7.6.42
id-MLDSA65-RSA4096-PSS-SHA512	1.3.6.1.5.5.7.6.43
id-MLDSA65-RSA4096-PKCS15-SHA512	1.3.6.1.5.5.7.6.44
id-MLDSA65-ECDSA-P256-SHA512	1.3.6.1.5.5.7.6.45
id-MLDSA65-ECDSA-P384-SHA512	1.3.6.1.5.5.7.6.46
id-MLDSA65-ECDSA-brainpoolP256r1-SHA512	1.3.6.1.5.5.7.6.47
id-MLDSA65-Ed25519-SHA512,	1.3.6.1.5.5.7.6
id-MLDSA87-ECDSA-P384-SHA512	1.3.6.1.5.5.7.6.49
id-MLDSA87-ECDSA-brainpoolP384r1-SHA512	1.3.6.1.5.5.7.6.50
id-MLDSA87-Ed448-SHAKE256,	1.3.6.1.5.5.7.6
id-MLDSA87-RSA3072-PSS-SHA512	1.3.6.1.5.5.7.6.52
id-MLDSA87-RSA4096-PSS-SHA512	1.3.6.1.5.5.7.6.53
id-MLDSA87-ECDSA-P521-SHA512	1.3.6.1.5.5.7.6.54
Dilithium2	1.3.6.1.4.1.2.267.7.4.4
Dilithium3	1.3.6.1.4.1.2.267.7.6.5
Dilithium5	1.3.6.1.4.1.2.267.7.8.7

Table C.6: Curated list of cryptography algorithm OIDs defined for IETF Hackathons [26]

D Code Excerpts

```
1 CREATE TABLE IF NOT EXISTS certificates (  
2     fingerprint_sha256 Nullable(String),  
3     fingerprint_sha1  Nullable(String),  
4     fingerprint_md5   Nullable(String),  
5     tbs_fingerprint_sha256 Nullable(String),  
6     tbs_no_ct_fingerprint_sha256 Nullable(String),  
7     spki_subject_fingerprint_sha256 Nullable(String),  
8     parent_spki_subject_fingerprint_sha256 Nullable(String),  
9     parsed JSON(  
10         version Int64,  
11         serial_number String,  
12         issuer_dn String,  
13         issuer Tuple(...),  
14         subject_dn String,  
15         subject Tuple(...),  
16         subject_key_info Tuple(...),  
17         validity_period Tuple(...),  
18         signature Tuple(...),  
19         extensions Tuple(  
20             key_usage Tuple(...),  
21             extended_key_usage Tuple(...),  
22             (...)  
23         ),  
24         unknown_extensions Array(...),  
25         redacted Bool,  
26         serial_number_hex String,  
27         ja4x String  
28     ),  
29     precert Bool,  
30     revoked Bool,  
31     names Array(String),  
32     validation_level String,  
33     validation JSON(...),  
34     revocation JSON(...),  
35     ct JSON(...),  
36     ever_seen_in_scan Bool,  
37     raw String,  
38     added_at Nullable(DateTime64(6)),  
39     modified_at DateTime64(6),  
40     validated_at Nullable(DateTime64(6)),  
41     parse_status String,  
42     zlint JSON(...),  
43     labels Array(String),  
44     not_valid_after Nullable(DateTime64(6)),  
45     inserted_at Nullable(DateTime64(6)),  
46     ct_hit Bool,  
47     certificate_type Enum('leaf', 'intermediate', 'root', 'unknown'),  
48     trust_status Enum('trusted', 'was-trusted', 'untrusted'),  
49     key_usage_set Array(String),  
50     extended_key_usage_set Array(String),  
51     alt_signature_algorithm Nullable(String)  
52 ) ENGINE = ReplacingMergeTree(modified_at)  
53 ORDER BY fingerprint_sha256  
54 SETTINGS allow_nullable_key = True
```

Listing D.1: Compact SQL statement used to create a table for persisting the certificates.